

1-CAT-Estructures_dades_avançades

July 1, 2021

1 Programació per a la ciència de dades

1.1 Unitat 1: Estructures de dades avançades en Python

1.1.1 Instruccions d'ús

Aquest document és un *notebook* interactiu que combina explicacions més aviat teòriques de conceptes de programació i fragments de codi executables. Per aprofitar els avantatges que aporta aquest format, us recomanem que, en primer lloc, llegiu les explicacions i el codi que us proporcionem. D'aquesta manera tindreu un primer contacte amb els conceptes que hi exposem. Ara bé, **la lectura és només el principi!** Una vegada hàgiu llegit el contingut proporcionat, no oblideu executar el codi proporcionat i modificar-lo per crear-ne variants que us permetin comprovar que n'heu entès la funcionalitat i explorar-ne els detalls d'implementació. Finalment, us recomanem també consultar la documentació enllaçada per explorar amb més profunditat les funcionalitats dels mòduls presentats.

```
[9]: %load_ext pycodestyle_magic
```

```
[10]: # Activem les alertes d'estil
      %pycodestyle_on
```

1.1.2 Introducció

En aquesta unitat es presenten estructures de dades i tipus de dades avançats i es fa una introducció a la manipulació avançada de cadenes de caràcters.

En primer lloc, s'expliquen estructures de dades avançades que amplien les estructures que ja coneixíem (l·listes, tuples i diccionaris). Veurem què són i com fer servir en Python els conjunts, les piles, les cues, els diccionaris amb ordre i amb valors per defecte.

En segon lloc, veurem tipus de dades complexos. En concret, presentarem tipus de dades que permeten representar dates i hores, i explicarem les funcions que Python implementa sobre aquests tipus.

En tercer lloc i per acabar, ens centrarem en la manipulació avançada de cadenes de caràcters repassant algunes de les funcions bàsiques sobre cadenes i introduint el llenguatge i ús d'expressions

regulars.

A continuació s'inclou la taula de continguts que podeu fer servir per navegar pel document:

- 1. Estructures de dades avançades
 - 1.1. Conjunts
 - 1.2. Piles i cues
 - 1.3. Diccionaris ordenats
 - 1.4. Altres variants de diccionari
 - 2. Tipus de dades avançats: el mòdul datetime
 - 2.1. Creació d'objectes que representen dates i hores
 - 2.2. Mostrar la data i el temps en diferents formats
 - 2.3. Treball amb dates i temps
 - 2.4. Increments de temps
 - 2.5. El calendari
 - 2.6. Recapitulació
 - 3. Cadenes de caràcters
 - 3.1. Manipulació de cadenes de caràcters
 - 3.2. Expressions regulars
 - 3.2.1. Funcions del mòdul re
 - 3.2.2. Sintaxi de les expressions regulars
 - 3.2.3. Ús d'expressions regulars en la manipulació de cadenes
 - 4. Exercicis per practicar
 - 4.1. Solucions dels exercicis per practicar
 - 5. Bibliografia
 - 5.1. Bibliografia bàsica
 - 5.2. Bibliografia addicional
-

2 1. Estructures de dades avançades

Fins ara hem vist algunes de les estructures de dades més bàsiques que es fan servir per programar en Python: les llistes, les tuples i els diccionaris. En aquesta unitat presentarem estructures de dades més complexes que ens permetran afrontar altres tipus de problemes, com els conjunts, les piles, les cues, els diccionaris amb ordre i els diccionaris amb valors per defecte.

2.1 1.1. Conjunts

De la mateixa manera que les llistes, els conjunts són una col·lecció d'elements possiblement **heterogenis** (és a dir, que poden ser de tipus diferents). Ara bé, a diferència de les llistes, els elements d'un conjunt **no tenen ordre** (ni, per tant, posició dins del conjunt). A més, un conjunt **no pot tenir elements repetits**. En essència, aquesta estructura de dades implementa el concepte matemàtic de conjunt.

Python disposa de la classe `set`, que implementa el conjunt. Podem crear un conjunt fent servir el constructor de la classe o bé utilitzant claus `{}`:

```
[3]: # Creem un conjunt fent servir dues sintaxis diferents
a_set = {"Beale ciphers", "Quipu", "Zimmermann Telegram", "Chaocipher"}
the_same_set = set(
    ["Beale ciphers", "Quipu", "Zimmermann Telegram", "Chaocipher"])

# Confirmem que els dos conjunts creats són iguals
print("Boths sets are equal: {}".format(a_set == the_same_set))
```

Boths sets are equal: True

Cal anar amb compte si fem servir claus per definir els conjunts, ja que també es fan servir per definir diccionaris. Si els conjunts i diccionaris tenen elements, la sintaxi ens en determinarà el tipus. En canvi, fer servir claus per definir una col·lecció buida derivarà en un diccionari buit:

```
[4]: # Definim dos diccionaris buits fent servir les dues sintaxis
an_empty_dict = {}
another_empty_dict = dict()
print("Using {} results in a {} equivalent to dict(): {}".format(
    type(an_empty_dict), an_empty_dict == another_empty_dict))

# Definim un conjunt buit
an_empty_set = set()
print("Creating an empty {}".format(type(an_empty_set)))
```

Using {} results in a <class 'dict'> equivalent to dict(): True

Creating an empty <class 'set'>

Els conjunts no tenen ordre i, per tant, dos conjunts són iguals si els seus elements també ho són:

```
[5]: # Creem un altre conjunt amb els mateixos elements, però fem servir una altra
# ordenació dels elements en crear el conjunt
another_same_set = {"Zimmermann Telegram", "Quipu",
    "Chaocipher", "Beale ciphers"}
print("Boths sets are equal: {}".format(a_set == another_same_set))
```

Boths sets are equal: True

Igual que les llistes, els conjunts disposen de l'operació `len`, que retorna el nombre d'elements del conjunt (la seva cardinalitat). Les operacions `add` i `remove` permeten afegir i eliminar elements d'un conjunt:

```
[6]: # Mostrem la cardinalitat del conjunt
print("The set is: {}".format(a_set))
print("The size of the set is: {}".format(len(a_set)))

# Eliminem un element del conjunt
a_set.remove("Quipu")
print("After removing 'Quipu', the set is: {}".format(a_set))

# Afegim un element al conjunt
a_set.add("The Magic Words are Squeamish Ossifrage")
print("After adding an element, the set is: {}".format(a_set))
```

The set is: {'Beale ciphers', 'Zimmermann Telegram', 'Chaocipher', 'Quipu'}
The size of the set is: 4
After removing 'Quipu', the set is: {'Beale ciphers', 'Zimmermann Telegram', 'Chaocipher'}
After adding an element, the set is: {'Zimmermann Telegram', 'Chaocipher', 'The Magic Words are Squeamish Ossifrage', 'Beale ciphers'}

Els conjunts no tenen elements repetits:

```
[7]: print("The size of the set is: {}".format(len(a_set)))

# Ara afegim un element ja existent al conjunt
a_set.add("Beale ciphers")

# La cardinalitat del conjunt no varia, ja que l'element ja
# pertanyia al conjunt
print("The size of the set after adding 'Beale ciphers' is: {}"
      .format(len(a_set)))
```

The size of the set is: 4
The size of the set after adding 'Beale ciphers' is: 4

Ja hem vist com la classe `set` implementa la comparació d'igualtat de conjunts. Addicionalment, també implementa les operacions bàsiques de conjunts (unió, intersecció i diferència) i la diferència simètrica (que retorna els elements que estan només en un dels dos conjunts).

A continuació veiem un exemple d'aquestes operacions sobre tres conjunts, que contenen els noms dels estudiants de programació en Python, Java, i C, d'una escola (assumirem que el nom identifica de manera única els estudiants).

```
[8]: # Creem els tres conjunts
students_python = {"Anna", "Helena", "Marta", "Pol"}
students_java = {"Anna", "Teresa", "Helena"}
students_c = {"Marc"}

# Calculem els alumnes que estudien Python i també Java
print("Python and Java:\t\t{}".format(
```

```

    students_python.intersection(students_java)
))

# Calculem els alumnes que estudien Python però no java
print("Python but not Java:\t\t{}".format(
    students_python.difference(students_java)
))

# Calculem els alumnes que estudien Python o Java, però no
# tots dos llenguatges
print("Python or Java, but not both:\t{}".format(
    students_python.symmetric_difference(students_java)
))

# Calculem els estudiants que té l'escola (independentment
# del llenguatge que estudien)
print("Any of the languages:\t\t{}".format(
    students_python.union(students_java).union(students_c)
))

```

```

Python and Java:           {'Helena', 'Anna'}
Python but not Java:       {'Pol', 'Marta'}
Python or Java, but not both: {'Pol', 'Marta', 'Teresa'}
Any of the languages:      {'Marc', 'Pol', 'Marta', 'Helena', 'Anna',
'Teresa'}

```

Noteu que els resultats de les operacions entre conjunts que hem vist en la cel·la anterior són també conjunts, de manera que podem encadenar operacions (com mostra l'últim exemple, on s'obtenen tots els estudiants de l'escola).

Equivalentment, podem fer servir els operands `|`, `&`, `-` i `^`:

```

[9]: print("Python and Java:\t\t{}".format(
    students_python & students_java))
print("Python but not Java:\t\t{}".format(
    students_python - students_java))
print("Python or Java, but not both:\t{}".format(
    students_python ^ students_java))
print("Any of the languages:\t\t{}".format(
    students_python | students_java | students_c))

```

```

Python and Java:           {'Helena', 'Anna'}
Python but not Java:       {'Pol', 'Marta'}
Python or Java, but not both: {'Pol', 'Marta', 'Teresa'}
Any of the languages:      {'Marc', 'Pol', 'Marta', 'Helena', 'Anna',
'Teresa'}

```

Els conjunts són iterables, de manera que els podem recórrer com hem vist en la unitat anterior:

```
[10]: # Recorrem el conjunt students_python
for element in students_python:
    print(element)
```

```
Helena
Anna
Pol
Marta
```

Fins ara hem presentat les propietats dels conjunts i les operacions principals que podem fer sobre aquests conjunts amb la classe `set`. A més, hem vist que els conjunts són l'estructura de dades a fer servir quan volem mantenir una col·lecció d'elements sense duplicats i en la qual l'ordre no té importància. Els conjunts, així mateix, es poden fer servir com a eina per resoldre de manera eficient un cert tipus de problemes.

D'una banda, els conjunts són una de les maneres més ràpides d'eliminar duplicats d'una llista:

```
[11]: # Definim una llista amb elements duplicats
a_list = ["Anna", "Helena", "Marta", "Pol", "Anna", "Teresa", "Helena"]
print("A list with duplicates:\t\t\t{}".format(a_list))

# Creem una nova llista sense duplicats fent un cast de la llista inicial
# a conjunt, i convertint el resultat en llista de nou
a_list_without_duplicates = list(set(a_list))
print("The same list without duplicates:\t{}".format(
    a_list_without_duplicates))
```

```
A list with duplicates:                ['Anna', 'Helena', 'Marta', 'Pol',
'Anna', 'Teresa', 'Helena']
The same list without duplicates:      ['Pol', 'Marta', 'Helena', 'Anna',
'Teresa']
```

La solució anterior és, a més d'eficient, senzilla i elegant, tal com ens recomana el [PEP20](#).

Els conjunts en Python també estan optimitzats per fer testos de pertinença, que ens permeten comprovar si un determinat element està present en un conjunt:

```
[12]: a_student = "Anna"

print("Python students:\t{}".format(students_python))
print("Anna studies Python?:\t{}\n".format(a_student in students_python))

print("C students:\t\t{}".format(students_c))
print("Anna studies C?:\t{}".format(a_student in students_c))
```

```
Python students:        {'Helena', 'Anna', 'Pol', 'Marta'}
Anna studies Python?:   True

C students:             {'Marc'}
Anna studies C?:       False
```

2.2 1.2. Piles i cues

Les piles i les cues són unes estructures de dades també similars a les llistes en què els elements tenen ordre i poden estar repetits. La característica que defineix les piles i les cues és que els elements només es poden inserir i eliminar des d'un dels extrems.

Una **pila** (en anglès *stack*) només permet afegir i esborrar elements al seu cim (en anglès parlem del *top*), és a dir, per la part superior. Les piles es coneixen també com a estructures FILO (de l'anglès *first in last out*), ja que el primer element que s'afegeix a una pila serà l'últim que en sortirà. Una analogia d'una pila en el món físic pot ser una pila de llibres que és dins d'una caixa: el primer llibre que traurem de la caixa serà el de dalt de tot de la pila, que serà l'últim que hàgim posat a la caixa.

Una **cua** (en anglès *queue*) només permet afegir elements al final i eliminar-ne del principi. Les cues es coneixen també com a estructures FIFO (de l'anglès *first in first out*), ja que el primer element que s'afegeix a una cua serà el primer a sortir-ne. En el nostre dia a dia trobem cues en diverses circumstàncies, per exemple, quan esperem el nostre torn a la carnisseria o per comprar entrades al cinema.

A diferència dels conjunts, que en Python representem amb el tipus propi **set**, per treballar amb piles i cues amb Python farem servir el tipus llista (**list**). Aquest tipus disposa d'un seguit d'operacions que permeten emular el comportament de piles i cues. Python també disposa d'una classe específica per gestionar cues, la classe **queue**, que es fa servir en les implementacions *multi-thread* que veurem més endavant en l'assignatura.

Per implementar una pila, treballarem sobre una llista fent servir únicament els mètodes **append** i **pop** per afegir-hi i treure'n elements:

```
[13]: def print_stack(s, msg_bef=None, msg_after=None):
    # Definim la funció auxiliar print_stack, que ens permetrà
    # visualitzar una pila
    if msg_bef:
        print(msg_bef)

    if len(s) == 0:
        max_len = 10
    else:
        max_len = max([len(e) for e in s]) + 4

    print("|" + " "*(max_len+2) + "|")
    for e in s[::-1]:
        print("| " + e + " "*(max_len-len(e)) + " |")
    print("|" + " "*(max_len+2) + "|")

    print(msg_after+"\n" if msg_after else "\n")
```

```
[14]: # Inicialitzem una llista buida, que farem servir com a pila
stack = []
print_stack(stack, "Initial stack:")
```

```
# Afegim tres elements a la pila i anem mostrant el contingut de
# la pila en cada moment
stack.append("The Art of Computer Programming")
print_stack(stack, "Stack after adding 1 element:")

stack.append("The Mythical Man-Month")
print_stack(stack, "Stack after adding a 2nd element:")

stack.append("Refactoring")
print_stack(stack, "Stack after adding a 3rd element:")
```

Initial stack:

```
|
|_____|
```

Stack after adding 1 element:

```
|
| The Art of Computer Programming |
|_____|
```

Stack after adding a 2nd element:

```
|
| The Mythical Man-Month         |
| The Art of Computer Programming |
|_____|
```

Stack after adding a 3rd element:

```
|
| Refactoring                     |
| The Mythical Man-Month         |
| The Art of Computer Programming |
|_____|
```

En afegir elements, es van apilant de manera que l'últim afegit queda al *top* de la pila. En l'exemple anterior, l'últim element afegit, *Refactoring*, queda al *top* de la pila.

```
[15]: # Eliminem un element de la pila i mostrem com queda la pila
e = stack.pop()
print_stack(stack, "Stack after deleting one element:",
             "The element removed was: {}".format(e))
```

Stack after deleting one element:

```
|
|_____|
```



```
| The Mythical Man-Month |
| The Art of Computer Programming |
| _____ |
The element removed was: Refactoring
```

En executar `pop`, s'elimina l'element del *top* de la pila, en aquest cas, *Refactoring*. L'últim element afegit és, doncs, el primer a sortir-ne.

```
[16]: # Eliminem dos elements més de la pila i mostrem com queda la pila
# en cada moment
e = stack.pop()
print_stack(stack, "Stack after deleting another element:",
            "The element removed was: {}".format(e))

e = stack.pop()
print_stack(stack, "Stack after deleting the last element:",
            "The element removed was: {}".format(e))
```

```
Stack after deleting another element:
|
| The Art of Computer Programming |
| _____ |
The element removed was: The Mythical Man-Month
```

```
Stack after deleting the last element:
|
| _____ |
The element removed was: The Art of Computer Programming
```

Si continuem eliminant elements, veurem com el primer element afegit a la pila, *The Art of Computer Programming*, és l'últim a sortir-ne (el comportament FILO, *first in last out*, que hem comentat).

Finalment, executar un `pop` sobre una pila buida generarà una excepció:

```
[17]: try:
        stack.pop()
    except IndexError as e:
        print(e)
```

pop from empty list

Per implementar una cua, treballarem també sobre una llista fent servir únicament els mètodes `append` i `pop(0)` per afegir-hi i treure'n elements:

```
[18]: def print_queue(s, msg_bef=None, msg_after=None):
        # Definim la funció auxiliar print_queue, que ens permetrà
        # visualitzar una cua
        if msg_bef:
```

```

    print(msg_bef)

    if len(s) == 0:
        max_len = 10
    else:
        max_len = sum([len(e)+2 for e in s]) + 1

    print("_"*(max_len+2)+"\n")
    print("  " + "  ".join(s))
    print("_"*(max_len+2))

    print(msg_after+"\n" if msg_after else "\n")

```

```

[19]: # Inicialitzem una llista buida, que farem servir com a cua
queue = []
print_queue(queue, "Initial queue:")

# Afegim tres elements a la cua i anem mostrant el contingut de
# la cua en cada moment
queue.append("A")
print_queue(queue, "Queue after adding 1 element:")

queue.append("B")
print_queue(queue, "Queue after adding a 2nd element:")

queue.append("C")
print_queue(queue, "Queue after adding a 3rd element:")

```

Initial queue:

Queue after adding 1 element:

A

Queue after adding a 2nd element:

A B

Queue after adding a 3rd element:

```
-----  
  
A B C  
  
-----
```

```
[20]: # Eliminem un element de la cua i mostrem com queda la cua  
e = queue.pop(0)  
print_queue(queue, "Queue after deleting one element:",  
            "The element removed was: {}".format(e))
```

Queue after deleting one element:

```
-----  
  
B C  
  
-----  
The element removed was: A
```

En executar `pop(0)`, s'elimina el primer element de la cua, en aquest cas A, que és el primer element que hi hem afegit. Així, doncs, la cua és una estructura FIFO (*first in first out*).

```
[21]: # Eliminem dos elements més de la cua i mostrem com queda la cua  
# en cada moment  
e = queue.pop(0)  
print_queue(queue, "Queue after deleting another element:",  
            "The element removed was: {}".format(e))  
  
e = queue.pop(0)  
print_queue(queue, "Queue after deleting the last element:",  
            "The element removed was: {}".format(e))
```

Queue after deleting another element:

```
-----  
  
C  
  
-----  
The element removed was: B
```

Queue after deleting the last element:

```
-----  
  
-----
```

The element removed was: C

L'últim element afegit és, doncs, també l'últim a sortir-ne (C).

De nou, si intentem eliminar un element sobre una cua buida es genera una excepció:

```
[22]: try:
        queue.pop(0)
    except IndexError as e:
        print(e)
```

pop from empty list

2.3 1.3. Diccionaris ordenats

Tot i que a partir de la versió 3.6 de Python els diccionaris preserven l'ordre d'inserció dels elements, l'estructura de dades no té en compte l'ordre dels elements a l'hora de fer comparacions. Així, dos diccionaris amb el mateix contingut inserit en ordre diferent són considerats iguals. És a dir, dos diccionaris són iguals si el seu contingut ho és, independentment de l'ordre en què s'hagi inserit aquest contingut:

```
[23]: {"a": 0, "b": 1} == {"b": 1, "a": 0}
```

[23]: True

```
[24]: from collections import OrderedDict
OrderedDict([("a", 0), ("b", 1)]) == OrderedDict([("b", 1), ("a", 0)])
```

[24]: False

```
[25]: # Creem dos diccionaris amb el mateix contingut en ordre diferent
dict_1 = {"a": 0, "b": 1}
dict_2 = {"b": 1, "a": 0}

# Mostrem el contingut dels diccionaris en l'ordre en què items() retorna
# els elements
print("dict_1:")
for k, v in dict_1.items():
    print("{}: {}".format(k, v))

print("\ndict_2:")
for k, v in dict_2.items():
    print("{}: {}".format(k, v))

# Comprovem la igualtat dels dos diccionaris
print("\ndict_1 and dict_2 are equal: {}".format(dict_1 == dict_2))
```

```
dict_1:
a: 0
```

```
b: 1
```

```
dict_2:
```

```
b: 1
```

```
a: 0
```

```
dict_1 and dict_2 are equal: True
```

Si, al contrari, volem que l'ordre dels elements d'un diccionari en determini la semblança amb un altre diccionari, podem fer servir un diccionari ordenat, implementat per la classe `OrderedDict` en Python, dins la llibreria `collections`.

```
[26]: from collections import OrderedDict

# Creem dos diccionaris ordenats amb el mateix contingut en ordre diferent
dict_1 = OrderedDict([("a", 0), ("b", 1)])
dict_2 = OrderedDict([("b", 1), ("a", 0)])

# Comprovem la igualtat dels dos diccionaris
print("\ndict_1 and dict_2 are equal: {}".format(dict_1 == dict_2))
```

```
dict_1 and dict_2 are equal: False
```

Més enllà de la diferència en la implementació de la igualtat, la classe `OrderedDict` també ens ofereix un parell de mètodes relatius a l'ordre dels elements que no teníem disponibles en els diccionaris: `reversed` i `move_to_end`.

```
[27]: # Mostrem el contingut de dict_2 en l'ordre original
print("\ndict_2:")
for k in dict_2:
    print("{}: {}".format(k, dict_2[k]))

# Mostrem el contingut de dict_2 en ordre invers
print("\ndict_2:")
for k in reversed(dict_2):
    print("{}: {}".format(k, dict_2[k]))
```

```
dict_2:
```

```
b: 1
```

```
a: 0
```

```
dict_2:
```

```
a: 0
```

```
b: 1
```

```
[28]: # Creem un diccionari ordenat
dict_4 = OrderedDict([("a", 0), ("b", 1), ("c", 2), ("d", 3)])
```

```
# Movem l'element de clau 'b' al final del diccionari
dict_4.move_to_end('b')

# Mostrem el diccionari modificat amb l'element 'b' al final
print(dict_4)
```

```
OrderedDict([('a', 0), ('c', 2), ('d', 3), ('b', 1)])
```

Com a apunt final, cal tenir en compte que, en general, un diccionari estàndard de Python és més eficient que un diccionari ordenat. Així, doncs, farem servir sempre diccionaris estàndards si no és que necessitem les funcionalitats d'un diccionari ordenat per a la lògica de la nostra aplicació.

2.4 1.4. Altres variants de diccionari

Una altra variant del diccionari que sovint és útil és el `defaultdict`. Aquest tipus de diccionari comparteix les propietats del diccionari tradicional (`dict`), però permet establir un valor per defecte en les entrades del diccionari no inicialitzades:

```
[29]: # Creem un diccionari a_trad_dict i intentem mostrar el valor d'una clau
# inexistent
a_trad_dict = {}
try:
    print(a_trad_dict["non_existing_key"])
except KeyError as e:
    print(e)
```

```
'non_existing_key'
```

```
[30]: from collections import defaultdict

# Creem un diccionari defaultdict amb valor per defecte una llista
# i intentem mostrar el valor d'una clau inexistent
a_defaultdict = defaultdict(list)
print(a_defaultdict["non_existing_key"])

def get_0():
    # Creem un diccionari defaultdict amb valor per defecte una funció
    # que retorna 0, i intentem mostrar el valor d'una clau inexistent
    return 0

another_defaultdict = defaultdict(get_0)
print(another_defaultdict["non_existing_key"])
```

```
[]
```

```
0
```

Els `defaultdict` poden ser útils per evitar haver d'inicialitzar les entrades d'un diccionari manualment.

Per exemple, suposem que tenim un text i volem obtenir una llista de les paraules agrupades segons la seva lletra inicial. Podríem fer servir un diccionari per emmagatzemar aquesta agrupació: la clau del diccionari emmagatzemaria la lletra inicial, i el valor de cada clau seria una llista de les paraules que comencen per aquella lletra. Així, només caldria tenir entrades per a les lletres que són inicials d'alguna de les paraules del text i no per a totes les possibles lletres inicials.

Una implementació bàsica podria fer servir un diccionari normal processant cada paraula una a una i comprovant en cada cas si ja tenim una entrada al diccionari per a la lletra inicial de la paraula:

```
[31]: words = "Beautiful is better than ugly. Explicit is better than implicit. "  
words += "Simple is better than complex."  
  
# Creem el diccionari  
words_by_first_letter = dict()  
  
# Iterem per a cada paraula del text  
for word in words.split(" "):  
    # Recuperem la primera lletra  
    first_letter = word[0]  
  
    if first_letter in words_by_first_letter:  
        # Si ja existeix l'entrada del diccionari, afegim la paraula  
        # a la llista  
        words_by_first_letter[first_letter].append(word)  
    else:  
        # Si encara no existeix cap paraula amb la mateixa inicial, creem  
        # l'entrada al diccionari amb la llista de paraules per a aquesta  
        # inicial i afegint-hi la paraula  
        words_by_first_letter[first_letter] = [word]  
  
# Mostrem el resultat  
print(words_by_first_letter)
```

```
{'B': ['Beautiful'], 'i': ['is', 'is', 'implicit.', 'is'], 'b': ['better',  
'better', 'better'], 't': ['than', 'than', 'than'], 'u': ['ugly.'], 'E':  
['Explicit'], 'S': ['Simple'], 'c': ['complex.']}
```

Ara observem com podem simplificar el codi fent servir un `defaultdict`:

```
[32]: # Creem el diccionari defaultdict  
words_by_first_letter = defaultdict(list)  
  
# Iterem per a cada paraula del text  
for word in words.split(" "):  
    # Recuperem la primera lletra  
    first_letter = word[0]
```

```
# Afegim la paraula a la llista de la inicial corresponent
words_by_first_letter[first_letter].append(word)

print(words_by_first_letter)
```

```
defaultdict(<class 'list'>, {'B': ['Beautiful'], 'i': ['is', 'is', 'implicit.',
'is'], 'b': ['better', 'better', 'better'], 't': ['than', 'than', 'than'], 'u':
['ugly.'], 'E': ['Explicit'], 'S': ['Simple'], 'c': ['complex.']})
```

Fixeu-vos que en aquest cas no ens cal inicialitzar manualment la llista.

3 2. Tipus de dades avançats: el mòdul `datetime`

Ja coneixem els tipus de dades bàsics més habituals en Python: `int` i `float`, que ens permeten representar valors numèrics; `bool`, per representar valors booleans; i `str`, per representar cadenes de caràcters. També hem fet servir tipus compostos, com ara `list`, `tuple`, `dict` i `set`. En aquesta secció, presentarem altres classes que permeten representar tipus més complexos, com ara dates i temps.

El mòdul `datetime` de Python ofereix eines per treballar amb dates i hores. Així, aquest mòdul ens proveeix d'eines per afrontar els problemes més habituals de la gestió de dates i hores, com la representació del temps en cadenes de caràcters amb formats diferents, càlculs relatius a calendaris, el càlcul d'increments de temps, conversions entre unitats de temps, la gestió de fusos horaris, etc. Tot això facilita enormement la feina de tractar conjunts de dades que contenen dates i hores.

A tall d'exemple, intenteu pensar com implementaríeu, amb les llibreries de Python que hem presentat fins ara, petits programes que permetessin calcular:

1. A partir d'una data concreta, el dia de la setmana (dilluns, dimarts,..., diumenge) que era, és o serà aquell dia.
2. Si els valors “2019-11-15” i “Friday, November 15, 2019” corresponen o no corresponen a la mateixa data.
3. Quin dia serà d'aquí a trenta dies.
4. Si l'any 2200 serà o no serà un any de traspàs.

Tot aquest tipus de problemes són fàcilment implementables amb les funcions que proporciona el mòdul `datetime`. Les classes principals d'aquest mòdul són `date`, que permet treballar amb dates; `time`, amb temps; `datetime`, amb dates completes que inclouen també el temps; i `timedelta`, que processa increments de temps.

3.1 2.1. Creació d'objectes que representen dates i hores

Podem crear objectes que representen dates, hores o dates amb hores especificant les diferents components de cada estructura manualment. L'exemple següent mostra com crear una data, una hora i una data amb hora fent servir aquesta estratègia:

```
[33]: import datetime
```



```

# Creem un objecte date que representa una data concreta
a_date = datetime.date(year=2017, month=10, day=1)
# De manera més concisa, podríem fer:
# a_date = datetime.date(2017, 1, 11)
print(a_date)

# Creem un objecte time que representa una hora
a_time = datetime.time(hour=17, minute=14, second=42)
# De manera més concisa, podríem fer:
# a_time = datetime.time(17, 14, 42)
print(a_time)

# Creem un objecte datetime, que representa una data amb temps
a_datetime = datetime.datetime(year=2017, month=10, day=1,
                                hour=17, minute=14, second=42)
# De manera més concisa, podríem fer:
# a_datetime = datetime.datetime(2017, 1, 11, 17, 14, 42)
print(a_datetime)

```

```

2017-10-01
17:14:42
2017-10-01 17:14:42

```

Ara bé, és molt habitual trobar conjunts de dades en què les dates i hores no estan pas desglossades en any, mes, dia, hora, minut i segon, sinó que es representen com a cadenes de caràcters, que poden seguir convencions diferents a l'hora d'expressar les dates i les hores. Així, ens caldrà poder crear els objectes a partir de cadenes de caràcters i necessitarem algun llenguatge per poder definir el format d'aquestes cadenes.

Les classes `datetime`, `date` i `time` disposen del mètode `strptime`, que permet crear un objecte `datetime`, `date` i `time` a partir d'una cadena de caràcters amb la data i/o hora i d'una altra cadena de caràcters que n'informa del format. A continuació s'exposa com crear un objecte `datetime` a partir d'una cadena de caràcters. Tota aquesta funcionalitat pot ser utilitzada també de manera anàloga per crear objectes `date` i `time`.

Així, doncs, podem crear un objecte `datetime` a partir de la cadena de caràcters "2017-10-01 17:14:42" de la manera següent:

```

[34]: str_date = "2017-10-01 17:14:42"
      a_datetime_from_str = datetime.datetime.strptime(str_date, "%Y-%m-%d %H:%M:%S")
      print(a_datetime_from_str)
      a_datetime == a_datetime_from_str

```

```

2017-10-01 17:14:42

```

```

[34]: True

```

El primer paràmetre de la crida, "2017-10-01 17:14:42", informa de la data a representar, mentre que el segon paràmetre, "%Y-%m-%d %H:%M:%S", informa del format que fa servir la primera

cadena. Per representar el format, es fan servir uns comodins, indicats amb el símbol %, i altres caràcters no especials (com ara el guionet, -, o els dos punts, :). Així, en aquest cas informem que la cadena que representa la data i el temps està formada per: * l'any representat fent servir quatre dígitos (%Y), * un guió que separa l'any del mes (-), * el mes representat fent servir dos dígitos (%m), * un guió que separa el mes del dia (-), * el dia representat fent servir dos dígitos (%d), * un espai que separa la data de l'hora (), * l'hora representada en format 24 hores i fent servir dos dígitos (%H), * els dos punts que separa l'hora dels minuts (:), * els minuts representats amb dos dígitos (%M), * els dos punts que separen els minuts dels segons (:), * i, finalment, els segons representats amb dos dígitos (%S).

És important tenir en compte que l'especificació del format ens permet evitar tota ambigüitat a l'hora d'interpretar la data. Així, per exemple, podem saber que es vol representar el dia 1 d'octubre i no pas el dia 10 de gener. Podríem canviar aquesta interpretació modificant la cadena que expressa el format, de manera que el mes i el dia s'intercanviessin l'ordre:

```
[35]: a_date_from_str = datetime.datetime.strptime(str_date, "%Y-%d-%m %H:%M:%S")
      print(a_date_from_str)
```

2017-01-10 17:14:42

Fixeu-vos que en l'exemple anterior la cadena que representa la data és exactament la mateixa, però canviant l'especificació del format hem fet que s'interpreti de manera diferent, de manera que ara indica 10 de gener.

La cadena de caràcters que hem fet servir sempre utilitza el format complet per expressar qualsevol unitat de temps, anteposant zeros si cal. Així, per representar el primer dia del mes, fa servir dos dígitos, 01. Hi ha comodins per representar també les diferents unitats sense anteposar el zero i per representar anys fent servir només dos dígitos, mesos fent servir el nom del mes en comptes del número, hores en format AM/PM i moltes altres variants! Podeu trobar l'especificació completa del format en la [documentació oficial del mòdul datetime](#). A continuació, s'exposen alguns exemples addicionals:

```
[36]: # Canviant el format dels separadors
a_date_from_str = datetime.datetime.strptime(
    "2017/10/01 17*14*42", "%Y/%m/%d %H*%M*%S")
print(a_date_from_str)
```

2017-10-01 17:14:42

```
[37]: # Expressant l'any amb dos dígitos
a_date_from_str = datetime.datetime.strptime(
    "17-10-01 17:14:42", "%y-%m-%d %H:%M:%S")
print(a_date_from_str)
a_date_from_str = datetime.datetime.strptime(
    "95-10-01 17:14:42", "%y-%m-%d %H:%M:%S")
print(a_date_from_str)
```

2017-10-01 17:14:42

1995-10-01 17:14:42

Noteu que en el cas anterior Python interpreta el segle en funció dels últims dos dígit de l'any.

```
[38]: # Incloent el dia de la setmana i expressant el mes amb el nom
a_date_from_str = datetime.datetime.strptime(
    'Sunday, 01 October 2017 17:14:42', "%A, %d %B %Y %H:%M:%S")
print(a_date_from_str)
```

2017-10-01 17:14:42

```
[39]: # Amb el dia de la setmana i el mes abreuiats
a_date_from_str = datetime.datetime.strptime(
    'Sun, 01 Oct 2017 17:14:42', "%a, %d %b %Y %H:%M:%S")
print(a_date_from_str)
```

2017-10-01 17:14:42

En informàtica, també es pot fer servir el *unix time stamp* com a manera de mesurar el temps. El *unix time stamp* representa el temps com un enter, que compta els segons que han passat des de l'1 de gener de 1970. Podem crear objectes que representin una data a partir d'un *unix timestamp* amb el mètode `fromtimestamp`:

```
[40]: a_date_from_ts = datetime.date.fromtimestamp(1573840426)
print(a_date_from_ts)
```

2019-11-15

Sovint també ens interessarà fer servir la data o temps actual en les nostres aplicacions. Podem crear objectes que representin el moment actual amb les funcions `today` i `now`:

```
[41]: # Creem un objecte amb la data actual
now_date = datetime.date.today()
print(now_date)

# Creem un objecte amb la data i hora actual
now_datetime = datetime.datetime.now()
print(now_datetime)
```

2020-11-09

2020-11-09 13:26:45.858642

3.2 2.2. Mostrar la data i el temps en diferents formats

Fins ara hem vist com el mètode `strptime` permet crear objectes `datetime`, `date` i `time` a partir d'una cadena de caràcters. De manera anàloga, el mètode `strftime` permet obtenir una cadena de caràcters que representa una data, hora o data amb hora en un format concret, que s'especifica fent servir la mateixa sintaxi que `strptime`.

```
[42]: # Creem un objecte amb la data i hora actual
now_datetime = datetime.datetime.now()
```

```

# Mostrem la data i hora fent servir quatre dígits per als anys i dos
# per al mes, dia, hora, minut i segon
print(now_datetime.strftime("%Y-%m-%d %H:%M:%S"))

# Invertim l'ordre del dia i el mes
print(now_datetime.strftime("%Y-%d-%m %H:%M:%S"))

# Canviant el format dels separadors
print(now_datetime.strftime("%Y/%m/%d %H*%M*%S"))

# Expressant l'any amb dos dígits
print(now_datetime.strftime("%y-%m-%d %H:%M:%S"))

# Incloent el dia de la setmana i expressant el mes amb el nom
print(now_datetime.strftime("%A, %d %B %Y %H:%M:%S"))

# Amb el dia de la setmana i el mes abreviats
print(now_datetime.strftime("%a, %d %b %Y %H:%M:%S"))

```

```

2020-11-09 13:26:46
2020-09-11 13:26:46
2020/11/09 13*26*46
20-11-09 13:26:46
Monday, 09 November 2020 13:26:46
Mon, 09 Nov 2020 13:26:46

```

A més de fer servir `strftime` per obtenir cadenes de caràcters que representin dates, també ens pot ser útil la funció `isoformat`, que ja retorna la cadena en format ISO 8601 (any - mes - dia, fent servir quatre dígits per a l'any i dos per al dia i mes), sense necessitat d'especificar-lo manualment:

```

[43]: # Mostrem la data i l'hora en format ISO 8601
now_datetime.isoformat()

```

```

[43]: '2020-11-09T13:26:46.383716'

```

3.3 2.3. Treballa amb dates i temps

Ja hem vist una de les funcionalitats dels objectes que representen dates i temps, que és interpretar i generar cadenes de caràcters que representen dates en diferents formats. A continuació, veurem altres operacions habituals amb aquests objectes.

Podem accedir a les diferents components que conformen una data amb hora consultant els atributs dels objectes `datetime`, `date` i `time`. Noteu que en aquest cas el tipus de dades dels valors als quals accedim és un enter en comptes d'una cadena de caràcters, com amb `strftime`:

```

[44]: # Accedim a les diferents components d'una data amb hora
y = now_datetime.year
mo = now_datetime.month

```

```

d = now_datetime.day
h = now_datetime.hour
mi = now_datetime.minute
s = now_datetime.second

print("Year is {} and is an {}".format(y, type(y)))
print("Month is {} and is an {}".format(mo, type(mo)))
print("Day is {} and is an {}".format(d, type(d)))
print("Hour is {} and is an {}".format(h, type(h)))
print("Minute is {} and is an {}".format(mi, type(mi)))
print("Second is {} and is an {}".format(s, type(s)))

```

```

Year is 2020 and is an <class 'int'>
Month is 11 and is an <class 'int'>
Day is 9 and is an <class 'int'>
Hour is 13 and is an <class 'int'>
Minute is 26 and is an <class 'int'>
Second is 46 and is an <class 'int'>

```

Més enllà de l'accés a les components que conformen la data, podem accedir a altres dades, com ara a quina setmana de l'any pertany o bé a quin dia de la setmana correspon:

```

[45]: # Obtenim una tupla de tres valors: (any ISO, número de setmana ISO,
# dia de la setmana ISO)
iso_cal = now_datetime.isocalendar()
print("The week is: {}".format(iso_cal[1]))
print("The day is: {}".format(iso_cal[2]))

```

```

The week is: 46
The day is: 1

```

```

[46]: # També podem obtenir únicament el dia de la setmana amb weekday()
print("The day is: {}".format(now_datetime.weekday()))

```

```

The day is: 0

```

Les dues alternatives fan servir convencions diferents per referir-se als dies de la setmana: `weekday` considera que el dilluns és un 0 i el diumenge és un 6, mentre que `isocalendar` representa el dilluns amb un 1 i el diumenge amb un 7.

Per convertir el valor numèric retornat per `weekday` a una cadena de caràcters amb el nom del dia de la setmana, podem fer servir el mòdul `calendar`:

```

[47]: # Importem el mòdul calendar
import calendar

# Obtenim el nom del dia
print('Day of Week: ', calendar.day_name[now_datetime.weekday()])

```

```
# Alternativament, també hauríem pogut fer servir strftime per obtenir la
# mateixa dada
print(now_datetime.strftime("Short day name: %a"))
print(now_datetime.strftime("Full day name: %A"))
```

Day of Week: Monday
Short day name: Mon
Full day name: Monday

La gestió de l'idioma en què es mostren els noms dels dies de la setmana o dels mesos es configura amb el mòdul `locale`. Així, per exemple, si volem obtenir el nom del dia de la setmana en anglès, farem:

```
[48]: # Importem el mòdul locale
import locale

# Especifiquem l'idioma anglès
locale.setlocale(locale.LC_ALL, 'en_US.UTF-8')

# Mostrem el nom del dia
print('Day of Week:', calendar.day_name[now_datetime.weekday()])
```

Day of Week: Monday

Una altra de les característiques que implementen els objectes de representació de dates i hores que treballem en aquesta unitat és la possibilitat de comparar-los:

```
[49]: # Importem la funció sleep, que permet introduir esperes en l'execució de codi
from time import sleep

# Obtenim la data i hora actuals
now_1_dt = datetime.datetime.now()
# Esperem tres segons
sleep(3)
# Obtenim la data i hora actuals (després de l'espera de tres segons)
now_2_dt = datetime.datetime.now()

# Comparem les dues dates
print('now_1_dt == now_2_dt: {}'.format(now_1_dt == now_2_dt))
print('now_1_dt < now_2_dt: {}'.format(now_1_dt < now_2_dt))
print('now_1_dt.date() == now_2_dt.date(): {}'.format(
    now_1_dt.date() == now_2_dt.date()))
```

now_1_dt == now_2_dt: False
now_1_dt < now_2_dt: True
now_1_dt.date() == now_2_dt.date(): True

Els dos valors de data i hora no són iguals (ja que hi ha tres segons de diferència), però, en canvi, el dia sí que és el mateix (a menys que executem el codi just en el moment en què canviem de dia).

3.4 2.4. Increments de temps

Una altra de les classes que incorpora el mòdul `datetime` és `timedelta`. Aquesta classe ens permet representar increments de temps i manipular dates i hores utilitzant aquests increments.

Fixem-nos, per exemple, en les dues dates que hem creat anteriorment amb diferència de tres segons. Hem vist que les podíem comparar per saber quina és anterior o si són iguals. Ara bé, com podríem saber el temps que ha transcorregut entre una i l'altra? Podríem calcular-lo manualment recuperant les diferents components de la data amb hora, però la classe `timedelta` ja implementa aquesta funcionalitat:

```
[50]: # Calculem el temps transcorregut i el mostrem
delta = now_2_dt - now_1_dt
print("Datetimes differ in: {} seconds".format(delta))
print("Delta is: {}".format(type(delta)))
```

```
Datetimes differ in: 0:00:03.003725 seconds
Delta is: <class 'datetime.timedelta'>
```

El resultat de restar dos objectes `datetime` és un objecte `timedelta` que representa el temps transcorregut. A continuació, veurem alguns altres exemples de com es fan servir objectes `timedelta` per representar increments de temps:

```
[51]: # Creem tres datetimes
dt_1 = datetime.datetime(year=2017, month=10, day=1,
                        hour=17, minute=14, second=42)

dt_2 = datetime.datetime(year=2017, month=12, day=1,
                        hour=17, minute=14, second=42)

dt_3 = datetime.datetime(year=2017, month=10, day=15,
                        hour=20, minute=16, second=42)

# Creem timedeltas amb les diferències
tm_1 = dt_2 - dt_1
tm_2 = dt_3 - dt_1
tm_3 = dt_3 - dt_2
print("Timedelta between dt_1 and dt_2: {}".format(tm_1))
print("Timedelta between dt_1 and dt_3: {}".format(tm_2))
print("Timedelta between dt_2 and dt_3: {}".format(tm_3))
```

```
Timedelta between dt_1 and dt_2: 61 days, 0:00:00
Timedelta between dt_1 and dt_3: 14 days, 3:02:00
Timedelta between dt_2 and dt_3: -47 days, 3:02:00
```

Fixeu-vos que en mostrar per pantalla el valor d'un `timedelta` obtenim una cadena de caràcters anotada (s'hi inclou el nombre de dies i la paraula `days`, seguida de l'hora informada amb `hora:minut:segons`).

Si ho preferim, també podem obtenir el nombre de segons que representa el `timedelta` com a valor numèric (en aquest cas un `float`):

```
[52]: # Mostrem el nombre de segons del tm_1
ts = tm_1.total_seconds()
print("Seconds elapsed: {}".format(ts))
print("Result is: {}".format(type(ts)))
```

```
Seconds elapsed: 5270400.0
Result is: <class 'float'>
```

També podem construir un objecte `timedelta` per obtenir dates passades o futures a partir d'una data de referència:

```
[53]: # Creem un timedelta de tres setmanes
d = datetime.timedelta(weeks=3)

# Calculem quin dia serà d'aquí a tres setmanes
future_3_weeks = now_1_dt + d
print("The date in three weeks will be: {}".format(future_3_weeks))

# Calculem quin dia era fa tres setmanes
past_3_weeks = now_1_dt - d
print("The date three weeks ago was: {}".format(past_3_weeks))
```

```
The date in three weeks will be: 2020-11-30 13:26:48.471583
The date three weeks ago was: 2020-10-19 13:26:48.471583
```

Es poden crear objectes `timedelta` especificant setmanes, dies, hores, minuts, segons, mil·lisegons o microsegons:

```
[54]: # Creem diversos timedeltas
one_day = datetime.timedelta(days=1)
eleven_seconds = datetime.timedelta(seconds=11)
five_hours = datetime.timedelta(hours=5)
eleven_and_so_seconds = datetime.timedelta(seconds=11.56845)
one_microsecond = datetime.timedelta(microseconds=1)

# Operem amb els timedeltas
print("One day in the future: {}".format(now_1_dt + one_day))
print("One day in the past: {}".format(now_1_dt - one_day))
print("Adding one microsecond: {}".format(now_1_dt + one_microsecond))
```

```
One day in the future: 2020-11-10 13:26:48.471583
One day in the past: 2020-11-08 13:26:48.471583
Adding one microsecond: 2020-11-09 13:26:48.471584
```

3.5 2.5. El calendari

En els exemples que hem vist en l'apartat anterior, hem fet servir el mòdul `calendar` per obtenir el nom del dia de la setmana corresponent a una data concreta. Les funcionalitats d'aquest mòdul, però, no es limiten a calcular el dia de la setmana. El mòdul `calendar`, com el seu nom indica, ens proveeix de funcionalitats relatives al calendari.

Un exemple de les funcionalitats més evidents d'un calendari és mostrar el mateix calendari.

```
[55]: # Mostrem el calendari de l'any 2020
print(calendar.calendar(2020))
```

```

                                2020

    January                     February                     March
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
      1  2  3  4  5                1  2                1
    6  7  8  9 10 11 12      3  4  5  6  7  8  9      2  3  4  5  6  7  8
   13 14 15 16 17 18 19      10 11 12 13 14 15 16      9 10 11 12 13 14 15
   20 21 22 23 24 25 26      17 18 19 20 21 22 23      16 17 18 19 20 21 22
   27 28 29 30 31           24 25 26 27 28 29           23 24 25 26 27 28 29
                                           30 31

    April                       May                           June
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
      1  2  3  4  5                1  2  3                1  2  3  4  5  6  7
    6  7  8  9 10 11 12      4  5  6  7  8  9 10      8  9 10 11 12 13 14
   13 14 15 16 17 18 19      11 12 13 14 15 16 17      15 16 17 18 19 20 21
   20 21 22 23 24 25 26      18 19 20 21 22 23 24      22 23 24 25 26 27 28
   27 28 29 30           25 26 27 28 29 30 31      29 30

    July                       August                       September
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
      1  2  3  4  5                1  2                1  2  3  4  5  6
    6  7  8  9 10 11 12      3  4  5  6  7  8  9      7  8  9 10 11 12 13
   13 14 15 16 17 18 19      10 11 12 13 14 15 16      14 15 16 17 18 19 20
   20 21 22 23 24 25 26      17 18 19 20 21 22 23      21 22 23 24 25 26 27
   27 28 29 30 31           24 25 26 27 28 29 30      28 29 30
                               31

    October                     November                     December
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
      1  2  3  4                1                      1  2  3  4  5  6
    5  6  7  8  9 10 11      2  3  4  5  6  7  8      7  8  9 10 11 12 13
   12 13 14 15 16 17 18      9 10 11 12 13 14 15      14 15 16 17 18 19 20
   19 20 21 22 23 24 25      16 17 18 19 20 21 22      21 22 23 24 25 26 27
   26 27 28 29 30 31        23 24 25 26 27 28 29      28 29 30 31
                               30
```

```
[56]: # Mostrem el calendari de l'octubre de 2020
print(calendar.month(2020, 10))
```

```

October 2020
Mo Tu We Th Fr Sa Su
```

```

        1  2  3  4
    5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31

```

Entre altres funcionalitats, el mòdul `calendar` ens permet saber si un any és o no és de traspàs amb la funció `isleap`, o bé comptar el nombre de dies de traspàs que hi ha entre dos anys:

```
[57]: # Calculem si l'any 2020 és de traspàs
calendar.isleap(2020)
```

```
[57]: True
```

```
[58]: # Calculem el nombre de dies de traspàs entre els anys
# 2000 i 2050
calendar.leapdays(2020, 2050)
```

```
[58]: 8
```

3.6 2.6. Recapitulant

A l'inici d'aquesta secció, ens hem plantejat un conjunt de preguntes relacionades amb la gestió de les dates i les hores que semblaven *a priori* poc fàcils de resoldre. Com a punt final de la secció, respondrem aquestes preguntes fent servir el que hem après en aquesta unitat i constatarem com pot ser de fàcil contestar-les amb les eines adequades.

En primer lloc, ens hem plantejat com obtenir el dia de la setmana (dilluns, dimarts,..., diumenge) corresponent a una data concreta.

A partir d'un objecte `date` (o `datetime`), que representa una data, hem presentat dues maneres diferents d'obtenir el dia de la setmana:

```
[59]: # Creem un objecte date que representa una data concreta
a_date = datetime.date(year=2022, month=12, day=25)

# Alternativa 1: Obtenim el dia amb .weekday() i el nom del dia
# amb el mètode day_name del mòdul calendar
print('Day of Week:', calendar.day_name[a_date.weekday()])

# Alternativa 2: Fem servir strftime per obtenir el nom del dia de la setmana
print(a_date.strftime("Day of Week: %A"))
```

```
Day of Week: Sunday
```

```
Day of Week: Sunday
```

En segon lloc, volíem descobrir si les cadenes de caràcters “2019-11-15” i “Friday, November 15, 2019” corresponien a la mateixa data o no.

Podem crear dos objectes `date` que representin les dates expressades per cadascuna de les cadenes de caràcters i, després, comparar-los per comprovar-ne la igualtat:

```
[60]: # Creem dos objectes date a partir de les cadenes de caràcters especificant
# l'anglès com a idioma per interpretar correctament la segona cadena
date_1 = datetime.datetime.strptime(
    "2019-11-15", "%Y-%m-%d")
locale.setlocale(locale.LC_ALL, 'en_GB.UTF-8')
date_2 = datetime.datetime.strptime(
    "Friday, November 15, 2019", "%A, %B %d, %Y")

# Comparem la igualtat de les dates creades
date_1 == date_2
```

[60]: True

En tercer lloc, volíem saber quin dia serà d'aquí a trenta dies.

Podem combinar la funcionalitat d'obtenció de la data actual amb un `timedelta` que representi trenta dies per tal de saber quin dia serà d'aquí a trenta dies:

```
[61]: # Creem un objecte amb la data actual
now_date = datetime.date.today()

# Creem un datetime de trenta dies
dt_30days = datetime.timedelta(days=30)

# Sumem els trenta dies a la data actual
print(now_date + dt_30days)
```

2020-12-09

Finalment, ens preguntàvem si l'any 2200 serà un any de traspàs o no.

El mòdul `calendar` ens permet comprovar amb el mètode `isleap` si un any és de traspàs:

```
[62]: # Comprovem si l'any 2200 és un any de traspàs
calendar.isleap(2200)
```

[62]: False

4 3. Cadenes de caràcters

Ja coneixem algunes de les funcions de manipulació de cadenes de caràcters bàsiques, que ens permeten concatenar cadenes, formar cadenes a partir d'altres cadenes i de variables numèriques, particionar cadenes (fent servir *slicing*) i crear cadenes de caràcters a partir de llistes. Iniciarem aquesta secció fent un repàs d'aquestes funcions bàsiques:

```
[63]: # Creem quatre cadenes de caràcters
str_1 = "Today is:"
str_2 = "Friday"
str_3 = "Coding day"
pep_20 = "Beautiful is better than ugly.\nExplicit is better than implicit."
```

```
[64]: # Creem noves cadenes concatenant les cadenes anteriors
# amb espais que les separen
str_4 = str_1 + " " + str_2
print(str_4)
str_5 = str_1 + " " + str_3
print(str_5)
```

Today is: Friday
Today is: Coding day

```
[65]: # Creem cadenes a partir de valors de variables numèriques
str_6 = str_1 + " my {}th day of programming".format(6)
print(str_6)
```

Today is: my 6th day of programming

```
[66]: # També podem fer servir la funcionalitat de les cadenes-f
# per formatar text de manera compacta
days = 6
f'{str_1} my {days}th day of programming'
```

```
[66]: 'Today is: my 6th day of programming'
```

```
[67]: # Mostrem els nou primers caràcters de la cadena pep_20
print(pep_20[:9])
# Mostrem els caràcters del 25 al 29 (aquest últim no inclòs)
print(pep_20[25:29])
# Mostrem l'últim caràcter
print(pep_20[-1:])
```

Beautiful
ugly
.

```
[68]: # Creem una cadena a partir d'una llista de cadenes
a_list_of_str = ["A", "B", "C", "D"]
"".join(a_list_of_str)
```

```
[68]: 'ABCD'
```

```
[69]: # Creem una cadena de caràcters amb els números del 0 al 9
      # separats per espais
      " ".join([str(x) for x in list(range(10))])
```

```
[69]: '0 1 2 3 4 5 6 7 8 9'
```

```
[70]: # Creem una cadena de caràcters amb els números del 0 al 9
      # separats per comes i espais
      ", ".join([str(x) for x in list(range(10))])
```

```
[70]: '0, 1, 2, 3, 4, 5, 6, 7, 8, 9'
```

4.1 3.1. Manipulació de cadenes de caràcters

A l'hora de processar una cadena de caràcters, sovint serà necessari particionar-la per tal de manipular-ne els fragments de manera individual. La funció `split` permet particionar una cadena de caràcters i obtenir com a resultat una llista de subcadenaes de la cadena original.

Cridada sense arguments, la funció `split` fa servir caràcters en blanc (espais, tabuladors, salts de línia, etc.) com a separadors, i retorna una llista de les subcadenaes separades, ometent aquests separadors:

```
[71]: # Separem la cadena pep_20
      print(pep_20)
      pep_20.split()
```

```
Beautiful is better than ugly.
Explicit is better than implicit.
```

```
[71]: ['Beautiful',
      'is',
      'better',
      'than',
      'ugly.',
      'Explicit',
      'is',
      'better',
      'than',
      'implicit.']
```

Fixeu-vos que les paraules ‘ugly.’ i ‘Explicit’ queden separades com a dos subcadenaes diferents, ja que el salt de línia entre l’una i l’altra s’interpreta com a separador. En canvi, si volem que se separin les paraules considerant únicament els espais en blanc, podem indicar-ho afegint l’argument `sep` a la crida:

```
[72]: # Separem la cadena pep_20 considerant només els espais
      # com a separadors
      pep_20.split(sep=" ")
```

```
[72]: ['Beautiful',
      'is',
      'better',
      'than',
      'ugly.\nExplicit',
      'is',
      'better',
      'than',
      'implicit.']
```

Podem fer servir qualsevol caràcter (o cadena) com a separador:

```
[73]: # Separem la cadena per les lletres "a"
      pep_20.split("a")
```

```
[73]: ['Be', 'utiful is better th', 'n ugly.\nExplicit is better th', 'n implicit.']
```

```
[74]: # Separem la cadena fent servir la cadena "better" com a separador
      pep_20.split("better")
```

```
[74]: ['Beautiful is ', ' than ugly.\nExplicit is ', ' than implicit.']
```

Una altra de les operacions de processament de cadenes d'ús comú és fer un recompte del nombre d'aparicions d'una subcadena dins d'una cadena:

```
[75]: # Comptem quantes vegades apareix la lletra "a" en la cadena pep_20
      pep_20_a_num = pep_20.count("a")
      print("PEP20 string has {} 'a'".format(pep_20_a_num))
      # Comptem quantes vegades apareix la subcadena "better" en la cadena pep_20
      pep_20_bet_num = pep_20.count("better")
      print("PEP20 string has {} 'better'".format(pep_20_bet_num))
      # Comptem quants espais en blanc té la cadena pep_20
      pep_20_ws_num = pep_20.count(" ")
      print("PEP20 string has {} whitespaces".format(pep_20_ws_num))
```

```
PEP20 string has 3 'a'
PEP20 string has 2 'better'
PEP20 string has 8 whitespaces
```

Hem vist com comptar el nombre d'aparicions d'una subcadena. Com ho fariem, però, si volguéssim saber on són aquestes aparicions? La funció `find` ens retorna l'índex en el qual s'inicia una subcadena o retorna `-1` si aquesta subcadena no existeix:

```
[76]: # Busquem l'índex de la primera aparició de 'beter'
      str_to_search = "beter"
      ind_better = pep_20.find(str_to_search)
      print("'beter' starts at position {}".format(ind_better))

      # Busquem l'índex de la primera aparició de 'better'
```

```
str_to_search = "better"
ind_better = pep_20.find(str_to_search)
print("'better' starts at position {}".format(ind_better))
```

```
'beter' starts at position -1
'better' starts at position 13
```

La cadena `better` apareix dues vegades dins de la cadena emmagatzemada en la variable `pep_20`. La funció `find` retorna l'índex de la primera aparició. Si volem trobar els índexs de la resta d'aparicions, caldrà indicar un segon paràmetre en la crida de `find` especificant la posició d'inici de la cerca:

```
[77]: # Busquem l'índex de la segona aparició de 'better'
ind2_better = pep_20.find(str_to_search, ind_better+1)
print("Next 'better' starts at position {}".format(ind2_better))
```

```
Next 'better' starts at position 43
```

```
[78]: # Busquem l'índex de la tercera aparició de 'better'
ind3_better = pep_20.find(str_to_search, ind2_better+1)
print("Next 'better' starts at position {}".format(ind3_better))
```

```
Next 'better' starts at position -1
```

Com que `better` només hi apareix dues vegades, la tercera vegada que fem la cerca la crida en `find` retorna `-1` i indica així que ja no hi ha més aparicions.

En algunes aplicacions no estarem interessats a comptar el nombre d'aparicions d'una subcadena ni a saber on és aquesta subcadena, sinó simplement a substituir-la per alguna altra cadena. En aquests casos, podem fer servir `replace`, que reemplaça totes les aparicions d'una subcadena per una altra cadena:

```
[79]: # Reemplacem els espais en blanc per guions
print(pep_20.replace(" ", "_"), "\n")
# Reemplacem 'better' per 'worse'
print(pep_20.replace("better", "worse"), "\n")
# Eliminem els espais en blanc reemplaçant-los per una cadena buida
print(pep_20.replace(" ", ""))
```

```
Beautiful_is_better_than_ugly.
Explicit_is_better_than_implicit.
```

```
Beautiful is worse than ugly.
Explicit is worse than implicit.
```

```
Beautifulisbetterthanugly.
Explicitisbetterthanimplicit.
```

En l'últim exemple s'eliminen tots els espais en blanc. A vegades, però, ens interessarà eliminar els espais en blanc que hi ha davant d'una cadena o darrere d'aquesta (o ambdós conjunts d'espais).

Per a aquestes situacions, trobem les funcions `lstrip` (eliminar els espais en blanc de l'esquerra o l'inici de la cadena), `rstrip` (eliminar els espais de la dreta o del final de la cadena) i `strip` (elimina els espais en blanc de davant i darrere de la cadena):

```
[80]: # Creem una cadena amb espais en blanc
str_with_spaces = "    A lot of    spaces "

# Mostrem el resultat d'aplicar rstrip, lstrip i strip a la cadena
print("String: #{}#\n".format(str_with_spaces))
print("rstrip: #{}#\n".format(str_with_spaces.rstrip()))
print("lstrip: #{}#\n".format(str_with_spaces.lstrip()))
print("strip: #{}#\n".format(str_with_spaces.strip()))
```

```
String: #    A lot of    spaces #
```

```
rstrip: #    A lot of    spaces#
```

```
lstrip: #A lot of    spaces #
```

```
strip: #A lot of    spaces#
```

És interessant tenir en compte que les funcions `strip`, `rstrip` i `lstrip` eliminen tots els espais en blanc que estan en la posició que processen (davant, darrere o en les dues posicions), però, en canvi, no tenen cap efecte sobre la resta d'espais de la cadena (per exemple, l'espai en blanc entre 'A' i 'lot' mai es veu afectat per l'execució d'aquestes funcions).

4.2 3.2. Expressions regulars

Malgrat que les funcions que hem vist fins ara són molt potents, es queden curtes per afrontar molts dels problemes que trobarem a l'hora de preprocessar dades que continguin cadenes de text.

Per exemple, si volguéssim separar una cadena partint-la per qualsevol vocal (en comptes de la lletra 'a' únicament, com en l'exemple anterior), com ho fariem? Amb el que hem vist fins ara, hauríem d'anar separant vocal a vocal, processant en cada cas totes les subcadenaes ja separades per les vocals anteriors.

Un altre exemple seria si volguéssim buscar no solament la paraula 'better' dins d'una cadena sinó qualsevol paraula de sis lletres, o potser buscar paraules que comencessin amb 'b' i acabessin amb 'tter', amb qualsevol lletra o lletres entre la 'b' i 'tter'. Com podríem implementar-ho amb les funcions que hem vist fins ara? Certament, seria possible, però el codi seria complex.

Per afrontar aquest tipus de problemes, s'acostumen a fer servir **expressions regulars** (també conegudes com a *regexes*, una contracció del terme anglès *regular expressions*). Una expressió regular és una expressió que fa servir un llenguatge especialitzat de descripció de cadenes de caràcters. Aquest llenguatge ens permet expressar amb facilitat condicions que descriuen com ha de ser una cadena, com ara les que hem exemplificat anteriorment (una cadena que representi qualsevol vocal, una cadena de sis lletres, una cadena que comenci amb 'b' i acabi amb 'tter', etc.). Després aquestes expressions es poden fer servir per a tasques com les que hem exposat anteriorment: buscar subcadenaes dins de cadenes, separar cadenes, substituir subcadenaes per altres subcadenaes, etc.

En aquest apartat, veurem, doncs, com podem fer servir expressions regulars per processar cadenes de caràcters. En primer lloc, veurem les funcions que ens ofereix el mòdul `re`. En segon lloc,

presentarem una introducció de la sintaxi de les expressions regulars.

4.2.1 3.2.1. Funcions del mòdul `re`

El mòdul `re` és la interfície del motor d'expressions regulars per a Python. Per tant, caldrà importar-lo quan vulguem treballar amb *regexes*.

El mòdul té funcions equivalents a les que hem vist anteriorment però que treballen amb expressions regulars en comptes de treballar amb cadenes de caràcters: * `split`: té el mateix comportament que el mètode `split` d'`str`, i, així, permet separar cadenes considerant un separador, que en aquest cas serà expressat amb una expressió regular. * `search`: té un comportament similar a `find` d'`str`, i, així, permet trobar la primera aparició d'una subcadena dins d'una cadena. En aquest cas la subcadena a buscar s'expressa com una expressió regular. * `sub`: té un comportament similar a `replace` d'`str`, i, així, permet reemplaçar subcadenaes per una altra cadena. En aquest cas la subcadena a reemplaçar s'expressa com una expressió regular.

Adicionalment, també incorpora altres funcions que són d'utilitat, com ara:

- `findall`, que permet buscar totes les aparicions (no encavalcades) d'una subcadena dins d'una altra cadena.

A continuació, veurem com podem fer servir aquestes funcions sense aprofitar la funcionalitat de les expressions regulars (que presentarem en l'apartat següent).

La funció `split` actua de manera anàloga a l'`split` d'`str`, de manera que retorna una llista amb les subcadenaes separades per un separador:

```
[81]: # Importem el mòdul re
import re

# Separem la cadena per les lletres "a"
re.split('a', pep_20)
```

```
[81]: ['Be', 'utiful is better th', 'n ugly.\nExplicit is better th', 'n implicit.']
```

Ara, però, la primera cadena (a) podrà ser també una expressió regular que ens descrigui com ha de ser el separador a fer servir.

La funció `search` busca la primera aparició d'una subcadena en una altra cadena. A diferència de `find`, no retorna directament l'índex d'inici de la coincidència sinó un objecte `match` que conté informació addicional sobre la coincidència:

```
[82]: # Busquem l'índex de la primera aparició de 'better'
str_to_search = "better"
srch_better = re.search(str_to_search, pep_20)
print("The returned object is: {}".format(type(srch_better)))
print("that contains: {}".format(srch_better))
print("Substring starts at {} and ends at {}".format(
    srch_better.start(), srch_better.end()))
```

```
The returned object is: <class 're.Match'>
that contains: <re.Match object; span=(13, 19), match='better'>
```

Substring starts at 13 and ends at 19

En canvi, la funció `findall` ens retornarà una llista amb totes les coincidències d'una expressió regular:

```
[83]: # Busquem totes les aparicions de 'better'
srch_better = re.findall(str_to_search, pep_20)
print("The returned object is: {}".format(type(srch_better)))
print("that contains: {}".format(srch_better))
```

```
The returned object is: <class 'list'>
that contains: ['better', 'better']
```

Finalment, la funció `sub` reemplaça totes les aparicions d'una subcadena per una altra:

```
[84]: # Reemplacem els espais en blanc per guions baixos
print(re.sub(" ", "_", pep_20))
```

```
Beautiful_is_better_than_ugly.
Explicit_is_better_than_implicit.
```

Fins ara hem vist que el mòdul `re` ens permet fer les mateixes operacions que havíem vist sobre cadenes de caràcters. Vegem ara com podem explotar el potencial d'aquest mòdul fent servir el llenguatge de les expressions regulars.

Les funcions que hem vist accepten un primer paràmetre que representa la subcadena a buscar i que es fa servir com a separador (en `split`), com a objectiu de la cerca (`search`, o `findall`) o bé com a cadena a reemplaçar (`sub`). Més enllà de ser una cadena de caràcters, aquest primer paràmetre pot fer servir el llenguatge de les expressions regulars per tal d'especificar condicions complexes sobre les cadenes a buscar.

4.2.2 3.2.2. Sintaxi de les expressions regulars

En una expressió regular distingim dos tipus de caràcters: * Caràcters **normals**, que representen el mateix caràcter. Per exemple, una 'a' és una expressió regular vàlida que representa una cadena de caràcters d'un sol caràcter, que és la mateixa lletra 'a'. * Caràcters **especials**, que serveixen per representar grups de caràcters o bé per explicar com cal interpretar les expressions que els envolten.

Així, per exemple, alguns caràcters especials es fan servir per representar **grups de caràcters**: * `.`: representa qualsevol caràcter (per defecte, qualsevol caràcter excepte la nova línia). * `\d`: representa qualsevol dígit decimal. * `\s`: representa qualsevol espai en blanc. * `\w`: representa caràcters que poden formar part d'una paraula (lletres majúscules i minúscules, números, el guionet baix). * `[]`: permet indicar un conjunt de caràcters, sia individualment (`[abc]` permetria representar les cadenes `a`, `b` o `c`) o com a rang (`[A-Z]` representa totes les lletres en majúscula).

Altres caràcters especials donen informació sobre **com interpretar** les expressions: * `|`: implementa l'operador OR i permet crear una expressió regular que coincideixi amb una expressió regular o bé amb una altra.

També hi ha un conjunt de caràcters especials que permeten especificar **repeticions** d'expressions regulars. Així, els caràcters següents indiquen les repeticions de l'expressió que les precedeix: * `*`: 0 o més aparicions. * `+`: 1 o més aparicions. * `?`: 0 o 1 aparició. * `{m}`: exactament *m* aparicions.

* {*m*,*n*}: entre *m* i *n* aparicions. Si un dels dos valors s'omet, només s'informa del mínim o màxim d'aparicions.

4.2.3 3.2.3. Ús d'expressions regulars en la manipulació de cadenes

Ara que ja coneixem les funcions de manipulació de cadenes del mòdul `re` i la sintaxi bàsica del llenguatge de les expressions regulars, vegem com podem explotar el potencial d'ambdues funcionalitats juntes amb un seguit d'exemples. En primer lloc, veurem com podem resoldre alguns dels problemes que hem comentat anteriorment. En segon lloc, veurem exemples addicionals d'ús d'expressions regulars.

Si volem separar una cadena de caràcters utilitzant qualsevol vocal com a separador, podem fer servir una expressió regular que representi totes les vocals i utilitzar-la amb la funció `split`. Els caràcters especials `[]` ens permeten expressar un conjunt de caràcters individuals, de manera que podem fer-los servir per expressar les vocals:

```
[85]: print(pep_20)

# Separem la cadena per qualsevol vocal
re.split(r'[aeiouAEIOU]', pep_20)
```

```
Beautiful is better than ugly.
Explicit is better than implicit.
```

```
[85]: ['B',
      '',
      '',
      't',
      'f',
      'l ',
      's b',
      'tt',
      'r th',
      'n ',
      'gly.\n',
      'xpl',
      'c',
      't ',
      's b',
      'tt',
      'r th',
      'n ',
      'mpl',
      'c',
      't.']
```

Fixeu-vos que la llista conté diverses cadenes buides, que resulten de separar vocals contigües. D'altra banda, és interessant tenir en compte que també fem servir el prefix `'r'` per indicar la cadena de caràcters que conforma l'expressió regular. El prefix `'r'` indica que la cadena s'ha d'interpretar

com a literal *en brut* (en anglès *raw string literal*).

Si volem separar les paraules podem fer servir el caràcter especial `\s` per tenir en compte qualsevol espai en blanc com a separador (tant l'espai com el salt de línia `\n` s'utilitzarien com a separador per a la cadena `pep_20`):

```
[86]: re.split(r'\s', pep_20)
```

```
[86]: ['Beautiful',  
      'is',  
      'better',  
      'than',  
      'ugly.',  
      'Explicit',  
      'is',  
      'better',  
      'than',  
      'implicit.']
```

En canvi, si volem separar les frases, podem fer servir com a separador un punt seguit d'un espai en blanc. Fixeu-vos que incloem l'espai en blanc després del punt, ja que si no l'última frase quedaria separada en dos pel punt final (la frase mateixa i una cadena buida) i el salt de línia quedaria inclòs en la segona frase:

```
[87]: # Separem la cadena per frases: les frases se separen per un punt  
      # El resultat inclou una cadena buida al final, ja que l'últim punt  
      # s'utilitza com a separador, i el salt de línia  
      re.split(r'\.', pep_20)
```

```
[87]: ['Beautiful is better than ugly', '\nExplicit is better than implicit', '']
```

```
[88]: # Separem la cadena per frases: les frases se separen per un punt seguit  
      # d'un espai en blanc  
      re.split(r'\.\s', pep_20)
```

```
[88]: ['Beautiful is better than ugly', 'Explicit is better than implicit.']
```

És important tenir en compte que per representar el punt hem fet servir l'expressió `\.` en comptes de `..`. El motiu és que, com hem vist, `.` és un caràcter especial que coincideix amb qualsevol caràcter. Si, al contrari, volem que l'expressió regular només coincideixi quan inclogui un punt (el caràcter `'.'`), caldrà **escapar** la seqüència fent servir la contrabarra (`\`).

Un altre exemple que ens hem plantejat és com buscar paraules que comencin amb `'b'` i acabin amb `'tter'`, amb qualsevol lletra o lletres entremig. Vegem com podem enfocar aquest exemple. En primer lloc, veurem com buscar paraules que comencin amb `'b'` i acabin amb `'tter'`, amb una única lletra entremig:

```
[89]: # Creem una variant de la cadena pep_20, que ens permetrà exemplificar  
      # els diferents casos
```

```

pep_20_bis = "Beautiful is better than ugly.\nExplicit is better than " \
             "implicit. Butter Much Webetter. bitter. bYYYtter or btter. python."

# Mostrem les cadenes d'exemple
print(pep_20)
print("\n")
print(pep_20_bis)

```

Beautiful is better than ugly.
Explicit is better than implicit.

Beautiful is better than ugly.
Explicit is better than implicit. Butter Much Webetter. bitter. bYYYtter or
btter. python.

Una primera aproximació seria definir una expressió regular que comencés amb ‘b’, següís amb qualsevol caràcter que pogués formar part d’una paraula (que podem expressar amb el caràcter especial \w), i acabés amb ‘tter’:

```

[90]: print(r"b\wtter in pep_20: {}".format(re.findall(r"b\wtter", pep_20)))
      print(r"b\wtter in pep_20_bis: {}".format(re.findall(r"b\wtter", pep_20_bis)))

```

```

b\wtter in pep_20: ['better', 'better']
b\wtter in pep_20_bis: ['better', 'better', 'better', 'bitter']

```

L’expressió sembla retornar els resultats esperats per a la cadena `pep_20`, però, en canvi, per a la cadena `pep_20_bis` ens retorna un ‘better’ addicional i no és capaç d’identificar ‘Butter’. Els motius d’aquest comportament són, d’una banda, que les expressions regulars són per defecte *case sensitive*, de manera que ‘B’ i ‘b’ es consideren cadenes diferents (i, per tant, ‘Butter’ no és una coincidència vàlida), i, d’altra banda, que no estem especificant delimitadors de les paraules, de manera que ‘Webetter’ és una coincidència vàlida per a ‘better’.

Si volem que s’ignorin si les lletres són majúscules o minúscules a l’hora d’avaluar una expressió regular, podem fer servir el *flag* `IGNORECASE` en la crida:

```

[91]: print(r"b\wtter in pep_20_bis: {}".format(
      re.findall(r"b\wtter", pep_20_bis, re.IGNORECASE)))

```

```

b\wtter in pep_20_bis: ['better', 'better', 'Butter', 'better', 'bitter']

```

D’aquesta manera, ‘Butter’ s’identifica correctament.

D’altra banda, si volem buscar paraules completes (que no formin part d’altres paraules) podem especificar el(s) caràcter(s) que esperem trobar davant i darrere de la paraula:

```

[92]: # Delimitem les paraules per espais en blanc
      print(r" b\wtter in pep_20_bis: {}".format(
            re.findall(r" b\wtter ", pep_20_bis, re.IGNORECASE)))
      # Delimitem les paraules per espais en blanc o bé punts

```

```
print(r"[ \.]b\wtter[ \.] in pep_20_bis: {}".format(
    re.findall(r"[ \.]b\wtter[ \.]", pep_20_bis, re.IGNORECASE)))
```

```
b\wtter in pep_20_bis: [' better ', ' better ', ' Butter ']
[ \.]b\wtter[ \.] in pep_20_bis: [' better ', ' better ', ' Butter ', '
bitter. ']
```

En segon lloc, veurem com buscar paraules que comencin amb ‘b’ i acabin amb ‘tter’, però acceptant més d’una lletra entremig. A partir de l’expressió que hem fet servir per a una sola lletra entre les dues subcadenaes, podem indicar que acceptem qualsevol nombre de repeticions de la lletra entre ‘b’ i ‘tter’ amb el modificador *, o bé exigir que n’hi hagi una com a mínim amb +:

```
[93]: print("Using *: {}".format(
        re.findall(r"[ \.]b\w*ttter[ \.]", pep_20_bis, re.IGNORECASE)))
print("Using +: {}".format(
        re.findall(r"[ \.]b\w+ttter[ \.]", pep_20_bis, re.IGNORECASE)))
```

```
Using *: [' better ', ' better ', ' Butter ', ' bitter.', ' bYYYttter ', '
bttter. ']
```

```
Using +: [' better ', ' better ', ' Butter ', ' bitter.', ' bYYYttter ']
```

Finalment, podríem fer servir una estratègia similar per identificar totes les paraules de sis lletres, delimitant-les per espais o punts i fent servir el caràcter especial \w, repetit exactament sis vegades:

```
[94]: # Repetint el caràcter \w sis vegades explícitament:
print("6 letter words: {}".format(
    re.findall(r"[ \.]\w\w\w\w\w\w[ \.]", pep_20_bis)))
# Especificant el nombre de repeticions amb {}:
print("6 letter words: {}".format(
    re.findall(r"[ \.]\w{6}[ \.]", pep_20_bis)))
```

```
6 letter words: [' better ', ' better ', ' Butter ', ' bitter.', ' python. ']
```

```
6 letter words: [' better ', ' better ', ' Butter ', ' bitter.', ' python. ']
```

Aquesta secció ha presentat una petita introducció a l’ús d’expressions regulars amb el mòdul `re` de Python. Ara bé, hi ha molts altres caràcters especials i construccions que es poden fer amb el llenguatge de les expressions regulars i que ens poden ser útils per processar cadenaes de caràcters. Per continuar l’aprenentatge de l’ús d’expressions regulars en Python, us recomanem llegir el *Regular Expression HOWTO* de la documentació oficial de Python.

Adicionalment, si voleu descobrir tots els detalls sobre el llenguatge d’expressions regulars i sobre les funcions que implementa el mòdul `re` de Python, podeu dirigir-vos a la pàgina oficial del [mòdul re](#).

5 4. Exercicis per practicar

A continuació, trobareu un conjunt de problemes que us poden servir per practicar els conceptes explicats en aquesta unitat. Us recomanem que intenteu fer aquests problemes vosaltres mateixos i que, una vegada fets, compareu la solució que us proposem amb la vostra. No dubteu a adreçar al

fòrum de l'aula tots els dubtes que sorgeixin de la resolució d'aquests exercicis o bé de les solucions proposades.

1. Decidiu quina és l'estructura de dades més adequada per respondre cadascuna de les preguntes següents i escriviu el codi que permeti respondre-les.

Disposem de dades d'un conjunt de ciutats del món. De cada ciutat sabem si té més de 14 milions d'habitants, si és capital del país on està i si té una densitat de població per sobre de 20.000 habitants per km^2 : * Les ciutats Xangai, Beijing, Delhi, Istanbul, Karachi, Canton i Kinshasa tenen més de 14 milions d'habitants. La resta de ciutats de les quals tenim dades no tenen més de 14 milions d'habitants. * Les ciutats Delhi, Beijing, Kinshasa, Tòquio, Moscou, Jakarta, Seül i el Caire són capitals del país on es troben. La resta de ciutats de les quals tenim dades no són capitals. * Les ciutats el Caire, Kinshasa, Delhi i Tòquio tenen una densitat de població de més de 20.000 habitants per km^2 . La resta de ciutats de les quals tenim dades no superen 20.000 habitants per km^2 .

1.1. De quantes ciutats (diferents) tenim dades? Assumirem que no hi ha cap ciutat que no compleixi com a mínim una de les propietats anteriors.

[95] : # Resposta

1.2. Quantes ciutats tenen més de 14 milions d'habitants i una densitat de població de més de 20.000 habitants per km^2 ?

[96] : # Resposta

1.3. Quines ciutats tenen una densitat de població de més de 20.000 habitants per km^2 però no més de 14 milions d'habitants?

[97] : # Resposta

1.4. Quin és el país amb un major nombre de ciutats de més de 14 milions d'habitants? Quantes ciutats d'aquestes característiques hi ha a cada país?

Per respondre aquestes preguntes, ens faltaria afegir informació al conjunt de dades de ciutats del qual disposem per fer l'activitat. Penseu quina seria l'estructura de dades més adequada per emmagatzemar aquesta informació extra i calculeu la resposta a la pregunta plantejada.

[98] : # Resposta

1.5. Quins són els dos països adjacents que tenen el major nombre de ciutats de més de 14 milions d'habitants?

Per respondre aquesta pregunta, ens faltaria afegir més informació al conjunt de dades de ciutats del qual disposem per fer l'activitat. Penseu quina seria l'estructura de dades més adequada per emmagatzemar aquesta informació extra i calculeu la resposta a la pregunta plantejada.

[99] : # Resposta

2. Calculeu quantes hores ha treballat la persona que ha escrit la frase següent:

"I started working at 17:22:42 and finished at 22:00:00"

```
[100]: sentence = "I started working at 17:22:42 and finished at 22:00:00"

# Resposta
```

3. Donada la cadena de caràcters **sentence**, reemplaça tots els espais en blanc per punts.

```
[101]: sentence = " From      time to time, Python makes an      incompatible change " \
               " to the      advertised semantics of core language constructs      "

# Resposta
```

4. Donada la mateixa cadena de caràcters de l'exercici anterior, reemplaça tots els espais en blanc contigus per un únic punt. És a dir, si trobeu tres espais en blanc consecutius, aquests s'han de reemplaçar per un únic punt i no per tres punts com hem implementat en l'exercici anterior. Elimineu els espais que estan a l'inici i al final de la cadena abans de substituir-los per punts.

```
[102]: # Resposta
```

5. Proporcioneu una llista amb totes les paraules de quatre lletres de la cadena de caràcters **sentence** que comencin per **t** o **F**.

```
[103]: # Resposta
```

6. Reemplaça totes les majúscules de la cadena **sentence** per interrogants.

```
[104]: # Resposta
```

7. Una aerolínia ens contracta per ajudar-la a optimitzar el procediment d'embarcament dels seus avions. L'aerolínia disposa de tres classes de bitllets: primera, *business* i turista. A l'hora d'embarcar, els clients se situen en tres cues, una per cada classe. Després, però, només hi ha dues hostesses que els validin la targeta d'embarcament, de manera que les tres cues inicials es converteixen en dues, a partir de les quals els passatgers accedeixen als avions.

Actualment l'aerolínia fa servir l'estratègia següent per convertir les tres cues inicials (per classe) en les dues cues que embarquen (una per cada hostessa):

D'una banda, els clients de primera i *business* van a parar a la cua 1 (cua prioritària), de manera que s'intercala un client de cada tipus a la cua prioritària sempre que hi hagi prou clients per fer-ho i després s'assignen tots els clients restants a la nova cua. Així, el primer client que s'assigna a la cua prioritària serà el primer client de la classe primera, el segon client de la cua prioritària serà el primer client de la cua de *business*, el tercer client de la cua prioritària serà el segon client de la classe primera, etc.

D'altra banda, els clients de classe turista van a parar a la cua 2 (cua no prioritària) seguint l'ordre que tenien a la cua de la classe turista. Ara bé, si un client de classe turista té mobilitat reduïda o va acompanyat d'infants, aquest se situa al capdavant de la cua prioritària. Si hi ha més d'un client en aquestes condicions, l'ordre que segueixen a la cua de la classe turista es manté.

Per tal d'avaluar com és de bona l'estratègia, l'aerolínia fa servir dues mètriques: 1. El temps que es tarda a embarcar l'avió són 30 segons per passatger considerant que les dues cues (1 i 2)

embarquen alhora. És a dir, si la cua 1 té un passatger i la cua 2 en té dos, es tardarà un minut a embarcar. 2. La satisfacció global dels seus clients, que es calcula fent la mitjana de la satisfacció de cada client, considerant que:

- Els clients de primera tenen una satisfacció de -25 vegades el nombre de posicions que han perdut a la cua prioritària respecte a la seva posició original a la cua de primera. És a dir, un client que estava en tercera posició a la cua de primera i que ocupa la cinquena posició de la cua prioritària, tindrà una satisfacció de -50 .
- Els clients de *business* sempre tenen una satisfacció de 0.
- Els clients amb bitllet de classe turista que han estat moguts a la cua prioritària tenen una satisfacció de 100. En canvi, els que han estat moguts a la cua no prioritària tenen una satisfacció de 25 si han avançat alguna posició a la cua 2 respecte a la seva posició inicial a la cua de la classe turista, o de 0 en qualsevol altre cas.

Com a analistes de dades, avaluarem la satisfacció i el temps de pujada a l'avió del vol 714.

7.1. Carregueu les dades dels passatgers del vol 714 que trobareu en el *dataset* `data/flight714.csv` i creeu les tres cues primera, *business* i turista amb les dades dels passatgers. Els clients estan ordenats segons la seva posició a la cua, i la columna `client_class` indica a quina de les cues pertanyen.

[105]: `# Resposta`

7.2. Mostreu quants passatgers hi ha a cada cua amb el detall de quants tenen criatures o bé mobilitat reduïda.

[106]: `# Resposta`

7.3. Implementeu una funció que generi les dues cues d'embarcament (cua prioritària i cua no prioritària) a partir de les tres cues obtingudes segons la classe del bitllet del passatger.

[107]: `# Resposta`

7.4. Implementeu una funció que calculi el temps que es tarda a embarcar l'avió i una funció que calculi la satisfacció dels passatgers.

[108]: `# Resposta`

7.5. Calculeu el temps que es tarda a embarcar el vol 714 i la satisfacció dels clients.

[109]: `# Resposta`

5.1 4.1. Solucions dels exercicis per practicar

1. Decidiu quina és l'estructura de dades més adequada per respondre cadascuna de les preguntes següents i escriviu el codi que permeti respondre-la.

Disposem de dades d'un conjunt de ciutats del món. De cada ciutat, en sabem si té més de 14 milions d'habitants, si és capital del país on està, i si té una densitat de població de més de 20.000 habitants per km^2 : * Les ciutats Xangai, Beijing, Delhi, Istanbul, Karachi, Canton i Kinshasa tenen més de 14 milions d'habitants. La resta de ciutats de les quals tenim dades no tenen més de

14 milions d'habitants. * Les ciutats Delhi, Beijing, Kinshasa, Tòquio, Moscou, Jakarta, Seül i el Caire són capitals del país on estan. La resta de ciutats de les quals tenim dades no són capitals. * Les ciutats el Caire, Kinshasa, Delhi i Tòquio tenen una densitat de població de més de 20.000 habitants per km^2 . La resta de ciutats de les quals tenim dades no superen 20.000 habitants per km^2 .

1.1. De quantes ciutats (diferents) tenim dades? Assumirem que no hi ha cap ciutat que no compleixi com a mínim una de les propietats anteriors.

```
[110]: # Resposta

# Podem desar les dades en tres conjunts, un per cadascuna de les propietats
# que volem analitzar, i calcular la unió d'aquests conjunts

more_than_14M = {"Xangai", "Beijing", "Delhi", "Istanbul", "Karachi",
                "Canton", "Kinshasa"}
capital = {"Delhi", "Beijing", "Kinshasa", "Tòquio", "Moscou", "Jakarta",
          "Seül", "el Caire"}
more_than_20K = {"el Caire", "Kinshasa", "Delhi", "Tòquio"}

print("We have data from {} different cities".format(
    len(more_than_14M.union(capital).union(more_than_20K))))
```

We have data from 12 different cities

1.2. Quantes ciutats tenen més de 14 milions d'habitants i una densitat de població de més de 20.000 habitants per km^2 ?

```
[111]: # Resposta

# Podem aprofitar els dos conjunts d'interès (more_than_14M i more_than_20K)
# creats a l'apartat anterior, i calcular-ne la intersecció.

print("There are {} cities with more than 14M inhabitants and a density "
      "over 20K/km^2".format(len(more_than_14M.intersection(more_than_20K))))
```

There are 2 cities with more than 14M inhabitants and a density over 20K/km²

1.3. Quines ciutats tenen una densitat de població de més de 20.000 habitants per km^2 però no més de 14 milions d'habitants?

```
[112]: # Resposta

# De nou, podem aprofitar els dos conjunts d'interès (more_than_14M i
# more_than_20K) creats en el primer apartat i calcular-ne ara la diferència.

print("Cities with a density over 20K/km^2 and less than 14M inhabitants "
      "different cities: {}".format(more_than_20K.difference(more_than_14M)))
```

Cities with a density over 20K/km² and less than 14M inhabitants different

```
cities: {'Tòquio', 'el Caire'}
```

1.4. Quin és el país amb un major nombre de ciutats de més de 14 milions d'habitants? Quantes ciutats d'aquestes característiques hi ha a cada país?

Per respondre aquestes preguntes, ens faltaria afegir informació al conjunt de dades de ciutats del qual disposem per fer l'activitat. Penseu quina seria l'estructura de dades més adequada per emmagatzemar aquesta informació extra i calculeu la resposta a la pregunta plantejada.

Resposta:

```
[113]: # Resposta

# Desem el país al qual pertany cadascuna de les ciutats amb més
# de 14 milions d'habitants
city_country = {'Shanghai': 'China', 'Beijing': 'China',
                'Delhi': 'India', 'Istanbul': 'Turkey',
                'Karachi': 'Pakistan', 'Guangzhou': 'China',
                'Kinshasa': 'Congo'}

# Comptem quantes ciutats de més de 14 milions d'habitants hi ha a cada país
cities_per_country = defaultdict(get_0)
for k, v in city_country.items():
    cities_per_country[v] += 1

print(cities_per_country)

# Recuperem el país amb més ciutats
# Opció 1: recorrent el diccionari amb un bucle
max_val = -1
city = None
for k, v in cities_per_country.items():
    if v > max_val:
        max_val = v
        city = k
print(city)
```

```
defaultdict(<function get_0 at 0x7fcab80a05e0>, {'China': 3, 'India': 1,
'Turkey': 1, 'Pakistan': 1, 'Congo': 1})
China
```

```
[114]: # Opció 2: recuperem la clau del diccionari amb valor màxim
import operator
print(max(cities_per_country.items(), key=operator.itemgetter(1))[0])
```

China

1.5. Quins són els dos països adjacents que tenen el major nombre de ciutats de més de 14 milions d'habitants?

Per respondre aquesta pregunta, ens faltaria afegir més informació al conjunt de dades de ciutats

del qual disposem per fer l'activitat. Penseu quina seria l'estructura de dades més adequada per emmagatzemar aquesta informació extra i calculeu la resposta a la pregunta plantejada.

```
[115]: # Resposta
```

```
cities_per_country.keys()
```

```
[115]: dict_keys(['China', 'India', 'Turkey', 'Pakistan', 'Congo'])
```

```
[116]: # Desem les fronteres de cada país
```

```
# (ometem els països que no estan al *dataset*)
```

```
country_frontier = {
```

```
    'China': ['India', 'Pakistan', 'Russia'],
```

```
    'India': ['China', 'Pakistan', 'Indonesia'],
```

```
    'Turkey': [],
```

```
    'Pakistan': ['China', 'India'],
```

```
    'Congo': []}
```

```
# Calculem quins són els dos països adjacents amb més ciutats
```

```
# de més de 14 milions d'habitants
```

```
cities_per_adj_countries = {}
```

```
countries = list(cities_per_country.keys())
```

```
for i in range(len(countries)):
```

```
    for j in range(i+1, len(countries)):
```

```
        country_1, country_2 = countries[i], countries[j]
```

```
        if country_1 in country_frontier[country_2]:
```

```
            cities_per_adj_countries[country_1 + '-' + country_2] = \
                cities_per_country[country_1] + cities_per_country[country_2]
```

```
cities_per_adj_countries
```

```
[116]: {'China-India': 4, 'China-Pakistan': 4, 'India-Pakistan': 2}
```

2. Calculeu quantes hores ha treballat la persona que ha escrit la frase següent:

"I started working at 17:22:42 and finished at 22:00:00"

```
[117]: sentence = "I started working at 17:22:42 and finished at 22:00:00"
```

```
# Resposta
```

```
start = datetime.datetime.strptime(sentence[21:29], "%H:%M:%S")
```

```
end = datetime.datetime.strptime(sentence[-8:], "%H:%M:%S")
```

```
working_time = end - start
```

```
print("The person has worked {} hours".format(working_time))
```

The person has worked 4:37:18 hours

3. Donada la cadena de caràcters `sentence`, reemplaça tots els espais en blanc per punts.

```
[118]: sentence = " From      time to time, Python makes an incompatible change " \
           " to the    advertised semantics of core language constructs  "

# Resposta

sentence.replace(" ", ".")
```

```
[118]: '.From...time.to.time,.Python.makes..an...incompatible.change..to.the...adver
tised.semantics.of.core.language.constructs...'
```

4. Donada la mateixa cadena de caràcters de l'exercici anterior, reemplaça tots els espais en blanc contigus per un únic punt. És a dir, si trobeu tres espais en blanc consecutius, aquests s'han de reemplaçar per un únic punt, i no per tres punts com hem implementat en l'exercici anterior. Elimineu els espais que estan a l'inici i al final de la cadena abans de substituir-los per punts.

```
[119]: # Resposta

# Eliminem els espais en blanc
stripped_sentence = sentence.strip()
# Substituïm els espais en blanc per punts
re.sub(" +", ".", stripped_sentence)
```

```
[119]: 'From.time.to.time,.Python.makes.an.incompatible.change.to.the.advertised.semanti
cs.of.core.language.constructs'
```

5. Proporcioneu una llista amb totes les paraules de quatre lletres de la cadena de caràcters `sentence` que comencin per `t` o `F`.

```
[120]: # Resposta

re.findall("[t|F]\w{3} ", sentence)
```

```
[120]: [' From ', ' time ']
```

3:19: W605 invalid escape sequence '\w'

6. Reemplaça totes les majúscules de la cadena `sentence` per interrogants.

```
[121]: # Resposta

re.sub("[A-Z]", "?", sentence)
```

```
[121]: ' ?rom      time to time, ?ython makes an incompatible change to the
advertised semantics of core language constructs  '
```

7. Una aerolínia ens contracta per ajudar-la a optimitzar el procediment d'embarcament dels seus avions. L'aerolínia disposa de tres classes de bitllets: primera, *business* i turista. A l'hora

d'embarcar, els clients se situen en tres cues, una per cada classe. Després, però, només hi ha dues hostesses que els validin la targeta d'embarcament, de manera que les tres cues inicials es converteixen en dues, a partir de les quals els passatgers accedeixen als avions.

Actualment l'aerolínia fa servir l'estratègia següent per convertir les tres cues inicials (per classe) en les dues cues que embarquen (una per cada hostessa):

D'una banda, els clients de primera i *business* van a parar a la cua 1 (cua prioritària), de manera que s'intercala un client de cada tipus a la cua prioritària sempre que hi hagi prou clients per fer-ho i després s'assignen tots els clients restants a la nova cua. Així, el primer client que s'assigna a la cua prioritària serà el primer client de la classe primera, el segon client de la cua prioritària serà el primer client de la cua de *business*, el tercer client de la cua prioritària serà el segon client de la classe primera, etc.

D'altra banda, els clients de classe turista van a parar a la cua 2 (cua no prioritària) seguint l'ordre que tenien a la cua de la classe turista. Ara bé, si un client de classe turista té mobilitat reduïda o va acompanyat d'infants, se situa al capdavant de la cua prioritària. Si hi ha més d'un client en aquestes condicions, l'ordre que segueixen a la cua de la classe turista es manté.

Per tal d'avaluar com és de bona l'estratègia, l'aerolínia fa servir dues mètriques: 1. El temps que es tarda a embarcar l'avió són 30 segons per passatger considerant que les dues cues (1 i 2) embarquen alhora. És a dir, si la cua 1 té un passatger i la cua 2 en té dos, es tardarà un minut a embarcar. 2. La satisfacció global dels seus clients, que es calcula fent la mitjana de la satisfacció de cada client, considerant que:

- Els clients de primera tenen una satisfacció de -25 vegades el nombre de posicions que han perdut a la cua prioritària respecte a la seva posició original a la cua de primera. És a dir, un client que estava en tercera posició a la cua de primera i que ocupa la cinquena posició de la cua prioritària tindrà una satisfacció de -50 .
- Els clients de *business* sempre tenen una satisfacció de 0.
- Els clients amb bitllet de classe turista que han estat moguts a la cua prioritària tenen una satisfacció de 100. En canvi, els que han estat moguts a la cua no prioritària tenen una satisfacció de 25 si han avançat alguna posició a la cua 2 respecte a la seva posició inicial a la cua de la classe turista, o de 0 en qualsevol altre cas.

Com a analistes de dades, avaluarem la satisfacció i el temps de pujada a l'avió del vol 714.

7.1 Carregueu les dades dels passatgers del vol 714 que trobareu en el *dataset* `data/flight714.csv` i creeu les tres cues primera, *business* i turista amb les dades dels passatgers. Els clients estan ordenats segons la seva posició a la cua, i la columna `client_class` indica a quina de les cues pertanyen.

```
[11]: # Resposta

import pandas as pd

df = pd.read_csv("data/flight714.csv")

bq, pq, tq = [], [], []
col_names = list(df.columns)
```

```

for i in range(len(df)):
    c = {c: df.iloc[i][c] for c in col_names}
    if c['client_class'] == "p":
        pq.append(c)
    if c['client_class'] == "b":
        bq.append(c)
    if c['client_class'] == "t":
        tq.append(c)

```

7.2 Mostreu quants passatgers hi ha a cada cua amb el detall de quants tenen criatures o bé mobilitat reduïda.

```

[12]: # Resposta

from copy import copy

def check_children_or_reduced(q):
    qcopy = copy(q)
    ctr = 0
    # Comptem quants passatgers tenen criatures o mobilitat reduïda
    while len(qcopy):
        e = qcopy.pop(0)
        if e['has_children'] or e['has_reduced_mobility']:
            ctr += 1
    return ctr

print("Passengers in first class: {}".format(len(pq)))
print("Passengers in business class: {}".format(len(bq)))
print("Passengers in tourist class: {}".format(len(tq)))

f_with_crm = check_children_or_reduced(pq)
b_with_crm = check_children_or_reduced(bq)
t_with_crm = check_children_or_reduced(tq)

print("Passengers in first class with children or reduced mob.: {}".format(f_with_crm))
print("Passengers in business class with children or reduced mob.: {}".format(b_with_crm))
print("Passengers in tourist class with children or reduced mob.: {}".format(t_with_crm))

```

```

Passengers in first class: 3
Passengers in business class: 10
Passengers in tourist class: 37
Passengers in first class with children or reduced mob.: 0
Passengers in business class with children or reduced mob.: 0

```

Passengers in tourist class with children or reduced mob.: 6

7.3. Implementeu una funció que generi les dues cues d'embarcament (cua prioritària i cua no prioritària) a partir de les tres cues obtingudes segons la classe del bitllet del passatger.

```
[13]: # Resposta

def compute_new_queues_alg_1(in_bq, in_pq, in_tq):
    # Copiem les dades de les cues d'entrada per no modificar els
    # valors de les variables d'entrada
    bq, pq, tq = copy(in_bq), copy(in_pq), copy(in_tq)
    # Inicialitzem les dues cues de sortida buides
    q1, q2 = [], []

    # Movem els passatgers de la classe turista
    while len(tq) != 0:
        client = tq.pop(0)
        if client["has_children"] or client["has_reduced_mobility"]:
            q1.append(client)
        else:
            q2.append(client)

    # Movem la resta de passatgers alternant-los entre primera i *business*
    while len(bq) != 0 or len(pq) != 0:
        if len(pq) != 0:
            client = pq.pop(0)
            q1.append(client)
        if len(bq) != 0:
            client = bq.pop(0)
            q1.append(client)

    return q1, q2
```

7.4 Implementeu una funció que calculi el temps que es tarda a embarcar l'avió i una funció que calculi la satisfacció dels passatgers.

```
[26]: # Resposta

def get_index_queue(e, q):
    """
    Obtain the position of the element e in the queue q.
    """
    qcopy = copy(q)
    idx = -1
    while len(qcopy):
        element = qcopy.pop(0)
        idx += 1
```



```

        if e == element:
            return idx
    return -1

def evaluate_time(q1, q2):
    time_per_pas = 30
    return max(len(q1), len(q2))*time_per_pas

def evaluate_sat(q1, q2, bq, pq, tq):

    num_pass = len(q1) + len(q2)

    # Tourists with children or reduced mobility
    q1_copy, t = copy(q1), 0
    while len(q1_copy):
        p = q1_copy.pop(0)
        if get_index_queue(p, tq) != -1: # p in tq
            t += 100

    # Other tourists
    q2_copy, t2 = copy(q2), 0
    while len(q2_copy):
        p = q2_copy.pop(0)
        if get_index_queue(p, q2) < get_index_queue(p, tq):
            t2 += 25

    # First class
    q1_copy, pr = copy(q1), 0
    while len(q1_copy):
        p = q1_copy.pop(0)
        # p in pq and q1.index(p) > pq.index(p)
        if get_index_queue(p, pq) != -1:
            if get_index_queue(p, q1) > get_index_queue(p, pq):
                pr += -25*(get_index_queue(p, q1)-get_index_queue(p, pq))

    return (t + t2 + pr)/num_pass

```

7.5 Calculeu el temps que es tarda a embarcar el vol 714 i la satisfacció dels clients.

```

[27]: # Resposta

q1, q2 = compute_new_queues_alg_1(bq, pq, tq)
print("Time: {}".format(evaluate_time(q1, q2)))
print("Satisfaction: {}".format(evaluate_sat(q1, q2, bq, pq, tq)))

```

Time: 930

Satisfaction: 6.0

6 5. Bibliografia

6.1 5.1. Bibliografia bàsica

Us recomanem revisar la documentació oficial de les funcions i classes descrites en aquesta unitat que trobareu enllaçades en cada un dels apartats que les descriuen.

A més, us recomanem revisar l'especificació completa del format d'`strftime` i `strptime` del mòdul `datetime` en la [documentació oficial del mòdul](#). No cal que conegueu tots els comodins que s'usen per descriure dates i hores, però una lectura de la documentació us proporcionarà una visió global de quin tipus d'expressions es poden construir.

De manera anàloga, us recomanem llegir el *Regular Expression HOWTO* de la documentació oficial de Python.

6.2 5.2. Bibliografia addicional (ampliació de coneixements)

En aquesta unitat hem presentat una petita introducció a l'ús d'expressions regulars amb el mòdul `re` de Python. Si voleu descobrir tots els detalls sobre el llenguatge d'expressions regulars i sobre les funcions que implementa el mòdul `re` de Python, podeu dirigir-vos a la pàgina oficial del [mòdul re](#). Si voleu practicar l'ús d'expressions regulars, us recomanem visitar [regex101](#) i [regexr](#), dues pàgines web que permeten provar expressions regulars amb diferents textos i que generen una explicació de l'expressió regular utilitzada. Addicionalment, per seguir practicant amb una mica de motivació addicional, us recomanem intentar completar el joc [slash-escape](#), un joc en el qual haureu de fer servir expressions regulars per aconseguir escapar.

D'altra banda, els exercicis resolts de la unitat fan ús de la funció `copy`. Si voleu més informació sobre aquesta funció i saber per què és necessari el seu ús en l'exemple, us recomanem llegir la documentació oficial del mòdul `copy` i aquesta [pregunta](#) d'stackoverflow. Intenteu reflexionar sobre per què en el cas de l'exemple hem utilitzat `copy` en comptes de `deepcopy`, i sobre si el codi funcionaria correctament usant `deepcopy`.